

Okapi — Source Code

Okapi

```
{-# LANGUAGE DuplicateRecordFields #-}
```

```
module Okapi
(
    Contract (..)
  , Signature
  , fn
  , serve
  , Server (..)
  , Client (..)

  , ParseError (..)
  , Result (..)

  , app
  , client
  , openApi

  , fetch
  , ClientError (..)
  , ClientSettings (..)

  , endpointToOpenApi

  , Request (..)
  , Response (..)

  , KnownMethod (..)
  , GET
  , POST
  , PUT
  , DELETE

  , KnownStatus (..)
  , S200
  , S201
  , S204
  , S404
  , S500

  , ArrayStyle (..)
```

```

, Cases (..)
, Responses
, getResponses

, IsoCodec (isoCodec)
, Value (..)
, (=.)
, IsoJson

, parseRequest
, parseRequestResult
, printRequest
, linkTo
, parseResponse
, parseResponseResult
, parseResponses
, printResponse

, Iso (..)
, Data (..)
, int, int16, int32, int64, integer
, bool, char, float, double, scientific, text
, day, localTime, utcTime, timeOfDay, uuid

, request, methodGET, methodPOST, methodDELETE
, method, path, query, pathOf, queryOf, headersOf

, response, status200, status201, status204, status404, status500

, segment, segment_, segments

, field, field', field_
, fieldStructured, fieldBareItem, fieldItem, fieldList, fieldDictionary
, contentType

, list, list'

, cookie, cookie', setCookie
, attribute, attribute', secure, httpOnly
) where

import Okapi.Codec (IsoCodec (..), ParseError (..), Result (..), Value (..), (=.)
import Okapi.Artifact.OpenApi (endpointToOpenApi, openApi)
import Okapi.Protocol.Request
  ( Request (..), method, path, query
  , pathOf, queryOf, headersOf, methodGET, methodPOST, methodDELETE, request
  )
import Okapi.Protocol.Body (IsoJson)
import Okapi.Data
  ( Iso (..), Data (..)
  , int, int16, int32, int64, integer
  , bool, char, float, double, scientific, text
  , day, localTime, utcTime, timeOfDay, uuid

```

```

    )
import Okapi.Protocol.Request.Method (DELETE, GET, KnownMethod (..), POST, PUT)
import Okapi.Protocol.Request.Query (ArrayStyle (..), list, list')
import Okapi.Protocol.Response
    ( Response (..), Cases (..), Responses, getResponses
    , response, status200, status201, status204, status404, status500
    )
import Okapi.Protocol.Response.Status (KnownStatus (..), S200, S201, S204, S404, S500)
import Okapi.Protocol.Request.Path (segment, segment_, segments)
import Okapi.Protocol.Headers
    ( field, field', field_, fieldStructured, fieldBareItem, fieldItem
    , fieldList, fieldDictionary, contentType, cookie, cookie', setCookie
    )
import Okapi.Protocol.Headers.Attributes (attribute, attribute', secure, httpOnly)
import Okapi.Contract (Signature, Contract (..), parseRequest, parseRequestResult, printRequest, linkTo, parseResponse)
import Okapi.Application.Server (Server (..), fn, serve, app)
import Okapi.Application.Client (Client (..), ClientError (..), ClientSettings (..), fetch, client)

```

Okapi.Application.Client

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Application.Client (
    ClientError (..),
    ClientSettings (..),
    Client (..),
    fetch,
    GClient,
    client,
) where

import Data.ByteString qualified as BS
import Data.ByteString.Lazy qualified as LBS
import Data.Kind (Type)
import Data.Text.Encoding (encodeUtf8)
import GHC.Generics
    ( D1, C1, S1, K1 (..), M1 (..), Rec0
    , Generic (..), Rep
    , (:*) (..)
    )
import Network.HTTP.Client qualified as HC
import Network.HTTP.Types qualified as HTTP
import Network.Wai qualified as Wai
import Okapi.Codec (Value (..))
import Okapi.Protocol.Request (Request)
import Okapi.Protocol.Response (Cases)
import Okapi.Contract (Signature)
import Okapi.Contract (Contract, parseResponses, printRequest)

data ClientError = ClientError deriving (Eq, Show)

data ClientSettings = ClientSettings
    { manager :: HC.Manager
    , baseUrl :: String
    }

data Client sig where
    Cb ::
        (Request Value method path query headers body -> IO (Either ClientError (responses Value))) ->
        Client (Signature method path query headers body responses)
```

```

fetch ::
  Cases responses =>
  HC.Manager ->
  String ->
  Contract (Signature method path query headers body responses) ->
  Request Value method path query headers body ->
  IO (Either ClientError (responses Value))
fetch mgr baseUrl endpoint reqVal = do
  waiReq <- printRequest endpoint reqVal
  hcReq <- toHCRequest baseUrl waiReq
  hcRes <- HC.httpLbs hcReq mgr
  either (const (Left ClientError)) Right <$> parseResponses endpoint (fromHCResponse hcRes)

toHCRequest :: String -> Wai.Request -> IO HC.Request
toHCRequest baseUrl waiReq = do
  body <- Wai.strictRequestBody waiReq
  base <- HC.parseUrlThrow baseUrl
  let pathBS = "/" <> BS.intercalate "/" (map encodeUtf8 (Wai.pathInfo waiReq))
      qs      = HTTP.renderQuery True (Wai.queryString waiReq)
  pure base
    { HC.method      = Wai.requestMethod waiReq
    , HC.path        = pathBS
    , HC.queryString = qs
    , HC.requestHeaders = Wai.requestHeaders waiReq
    , HC.requestBody  = HC.RequestBodyLBS body
    , HC.checkResponse = \_ _ -> pure ()
    }

fromHCResponse :: HC.Response LBS.ByteString -> Wai.Response
fromHCResponse hcRes = Wai.responseLBS
  (HC.responseStatus hcRes)
  (HC.responseHeaders hcRes)
  (HC.responseBody hcRes)

class GClient (epF :: Type -> Type) (clF :: Type -> Type) where
  gClient :: ClientSettings -> epF () -> clF ()

instance GClient epF clF => GClient (D1 dm epF) (D1 dm' clF) where
  gClient s (M1 ep) = M1 (gClient @epF @clF s ep)

instance GClient epF clF => GClient (C1 cm epF) (C1 cm' clF) where
  gClient s (M1 ep) = M1 (gClient @epF @clF s ep)

instance (GClient epL clL, GClient epR clR)
=> GClient (epL *: epR) (clL *: clR) where
  gClient s (epL *: epR) =
    gClient @epL @clL s epL *: gClient @epR @clR s epR

instance Cases responses => GClient
  (S1 sm (Rec0 (Contract (Signature method path query headers body responses))))
  (S1 sm' (Rec0 (Client (Signature method path query headers body responses)))) where
  gClient (ClientSettings mgr url) (M1 (K1 ep)) =
    M1 (K1 (Cb \reqVal -> fetch mgr url ep reqVal))

```

```

client ::
  forall server.
    ( Generic (server Contract)
    , Generic (server Client)
    , GClient (Rep (server Contract)) (Rep (server Client))
    ) =>
    server Contract ->
    ClientSettings ->
    server Client
client endpoints settings =
  to (gClient @(Rep (server Contract)) @(Rep (server Client)) settings (from endpoints))

```

Okapi.Application.Server

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Application.Server (
  Server (..),
  fn,
  type (~>),
  serve,
  tryServe,
  GServer,
  app,
) where

import Data.Kind (Type)
import GHC.Generics
  ( D1, C1, S1, K1 (..), M1 (..), Rec0
  , Generic (..)
  , Rep
  , (:*) (..)
  )
import Network.HTTP.Types qualified as HTTP
import Network.Wai qualified as Wai
import Okapi.Codec (Value)
import Okapi.Protocol.Request (Request)
import Okapi.Protocol.Response (Cases)
import Okapi.Contract (Signature)
import Okapi.Contract (Contract, parseRequest, printResponse)

data Server (n :: Type -> Type) sig where
  Fn ::
    ((Request Value method path query headers body, Wai.Request) -> n (responses Value)) ->
    Server n (Signature method path query headers body responses)

fn ::
  ((Request Value method path query headers body, Wai.Request) -> n (responses Value)) ->
  Server n (Signature method path query headers body responses)
fn = Fn

type (~>) :: (Type -> Type) -> (Type -> Type) -> Type
type f ~> g = forall a. f a -> g a
```

```

serve ::
  Cases responses =>
  (n ~> IO) ->
  Contract (Signature method path query headers body responses) ->
  Server n (Signature method path query headers body responses) ->
  Wai.Application
serve runner endpoint (Fn handler) waiReq respond = do
  parsed <- parseRequest endpoint waiReq
  case parsed of
    Left _    -> respond (Wai.responseLBS HTTP.status400 [] mempty)
    Right req -> do
      resVal <- runner (handler (req, waiReq))
      waiRes <- printResponse endpoint resVal
      respond waiRes

tryServe ::
  Cases responses =>
  (n ~> IO) ->
  Contract (Signature method path query headers body responses) ->
  Server n (Signature method path query headers body responses) ->
  Wai.Middleware
tryServe runner endpoint (Fn handler) next waiReq respond = do
  parsed <- parseRequest endpoint waiReq
  case parsed of
    Left _    -> next waiReq respond
    Right req -> do
      resVal <- runner (handler (req, waiReq))
      waiRes <- printResponse endpoint resVal
      respond waiRes

class GServer (n :: Type -> Type) (epF :: Type -> Type) (svF :: Type -> Type) where
  gServe :: (n ~> IO) -> epF () -> svF () -> Wai.Middleware

instance GServer n epF svF => GServer n (D1 dm epF) (D1 dm' svF) where
  gServe runner (M1 ep) (M1 sv) = gServe @n @epF @svF runner ep sv

instance GServer n epF svF => GServer n (C1 cm epF) (C1 cm' svF) where
  gServe runner (M1 ep) (M1 sv) = gServe @n @epF @svF runner ep sv

instance (GServer n epL svL, GServer n epR svR)
=> GServer n (epL :+: epR) (svL :+: svR) where
  gServe runner (epL :+: epR) (svL :+: svR) =
    gServe @n @epL @svL runner epL svL
    . gServe @n @epR @svR runner epR svR

instance Cases responses => GServer n
  (S1 sm (Rec0 (Contract (Signature method path query headers body responses))))
  (S1 sm' (Rec0 (Server n (Signature method path query headers body responses)))) where
  gServe runner (M1 (K1 ep)) (M1 (K1 sv)) = tryServe runner ep sv

app ::
  forall server n.
  ( Generic (server Contract)
  , Generic (server (Server n))

```



```

    , GServer n (Rep (server Contract)) (Rep (server (Server n)))
  ) =>
  server Contract ->
  (n ~> IO) ->
  server (Server n) ->
  Wai.Application
app endpoints runner handlers =
  gServe @n @(Rep (server Contract)) @(Rep (server (Server n)))
    runner
    (from endpoints)
    (from handlers)
  $ \_ respond -> respond (Wai.responseLBS HTTP.status404 [] mempty)

```

Okapi.Artifact.OpenApi

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE OverloadedLabels #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Artifact.OpenApi (endpointToOpenApi, GenericOAPI, openApi) where

import Control.Applicative ((<|>))
import Data.ByteString (ByteString)
import Data.ByteString.Char8 qualified as BS8
import Data.CaseInsensitive qualified as CI
import Data.Function ((&))
import Data.HashMap.Strict.InsOrd qualified as IHM
import Data.Kind (Type)
import Data.List (intercalate)
import Data.Maybe (fromMaybe)
import Data.OpenApi (OpenApi, Operation, Param, ParamLocation (..), PathItem (..), Referenced (..), Response, decl)
import Data.OpenApi qualified as OA
import Data.OpenApi.Declare (execDeclare)
import Data.OpenApi.Optics ()
import Data.Proxy (Proxy (..))
import Data.String (fromString)
import Data.Text (Text)
import Data.Text qualified as T
import GHC.Generics (C1, D1, Generic (..), K1 (..), M1 (..), Rec0, Rep, S1, (:*) (..))
import Network.HTTP.Media (MediaType)
import Network.HTTP.Types qualified as HTTP
import Okapi.Codec (Codec (..), IsoCodec (..))
import Okapi.Contract (Contract (..), Signature)
import Okapi.Protocol.Body (Body (..), IsoJson)
import Okapi.Protocol.Body qualified as Body
import Okapi.Data (Info (..), Iso (..))
import Okapi.Protocol.Headers (Headers (..))
import Okapi.Protocol.Headers qualified as H
import Okapi.Protocol.Request (Request (..))
import Optics.Core ((%), (.%), (?~))

import Okapi.Protocol.Request.Method qualified as Method
import Okapi.Protocol.Request.Path (Path)
import Okapi.Protocol.Request.Path qualified as Path
import Okapi.Protocol.Request.Query (Query)
import Okapi.Protocol.Request.Query qualified as Query

import Okapi.Protocol.Response (Cases, Responses (Responses), traverseResponses)
import Okapi.Protocol.Response qualified as ORes
import Okapi.Protocol.Response.Status qualified as Status
```

```

import Data.Functor.Const (Const (..), getConst)
import Data.List.NonEmpty qualified as NE

data PathPiece = PLit Text | PParam Text OA.Schema

walkPath :: Codec Path i o -> [PathPiece] -> [PathPiece]
walkPath (Lift (Path.Seg_ vIso x)) ps = ps ++ [PLit (vIso.encode x)]
walkPath (Lift (Path.Seg n vIso)) ps = ps ++ [PParam n (infoSchema vIso.info)]
walkPath (Lift (Path.Segs vIso)) ps = ps ++ [PParam "segs" (infoSchema vIso.info)]
walkPath (Lift Path.Raw) ps = ps
walkPath (FMap _ c) ps = walkPath c ps
walkPath (LMap _ c) ps = walkPath c ps
walkPath (Apply cf cx) ps = walkPath cx (walkPath cf ps)
walkPath (Pure _) ps = ps

pathTemplate :: [PathPiece] -> FilePath
pathTemplate pieces = "/" <> intercalate "/" (map piece pieces)
  where
    piece (PLit t) = T.unpack t
    piece (PParam n _) = "{" <> T.unpack n <> "}"

pathOAParams :: [PathPiece] -> [Param]
pathOAParams pieces =
  [ mkParamWithSchema name ParamPath True sc
  | PParam name sc <- pieces
  ]

extractQueryParams :: Codec Query i o -> [Param]
extractQueryParams (Lift (Query.Param key vIso)) = [mkParamWithSchema key ParamQuery True (infoSchema vIso.info)]
extractQueryParams (Lift (Query.Param' key vIso)) = [mkParamWithSchema key ParamQuery False (infoSchema vIso.info)]
extractQueryParams (Lift (Query.Param_ key vIso _)) = [mkParamWithSchema key ParamQuery True (infoSchema vIso.info)]
extractQueryParams (Lift (Query.Flag key)) = [mkParam key ParamQuery True]
extractQueryParams (Lift (Query.Flag' key)) = [mkParam key ParamQuery False]
extractQueryParams (Lift (Query.List style key _)) = [mkArrayParam key True style]
extractQueryParams (Lift (Query.List' style key _)) = [mkArrayParam key False style]
extractQueryParams (Lift Query.Raw) = []
extractQueryParams (FMap _ c) = extractQueryParams c
extractQueryParams (LMap _ c) = extractQueryParams c
extractQueryParams (Apply cf cx) = extractQueryParams cf ++ extractQueryParams cx
extractQueryParams (Pure _) = []

infoSchema :: Info -> OA.Schema
infoSchema (Info ty fmt) = withFmt (mempty & #type ?~ oaType ty)
  where
    oaType "integer" = OA.OpenApiInteger
    oaType "number" = OA.OpenApiNumber
    oaType "boolean" = OA.OpenApiBoolean
    oaType _ = OA.OpenApiString
    withFmt s = maybe s (\f -> s & #format ?~ f) fmt

extractHeaderParams :: Codec (Headers ctx) i o -> [Param]
extractHeaderParams (Lift hdr) = case hdr of
  H.Field key vIso -> [mkParamWithSchema (hdrName key) ParamHeader True (infoSchema vIso.info)]

```

```

H.Field' key vIso      -> [mkParamWithSchema (hdrName key) ParamHeader False (infoSchema vIso.info)]
H.Raw                  -> []
H.Field_ _ _          -> []
H.FieldStructured n _ -> [mkParamWithSchema (hdrName n) ParamHeader True (mempty & #type ?~ OA.OpenApiString)]
H.Cookie{}             -> []
H.Cookie'{}            -> []
H.SetCookie{}          -> []
extractHeaderParams (FMap _ c) = extractHeaderParams c
extractHeaderParams (LMap _ c) = extractHeaderParams c
extractHeaderParams (Apply cf cx) = extractHeaderParams cf ++ extractHeaderParams cx
extractHeaderParams (Pure _) = []

extractResHeaders :: Codec (Headers ctx) i o -> [(Text, Bool, OA.Schema)]
extractResHeaders (Lift hdr) = case hdr of
  H.Field key vIso      -> [(hdrName key, True, infoSchema vIso.info)]
  H.Field' key vIso     -> [(hdrName key, False, infoSchema vIso.info)]
  H.Raw                  -> []
  H.Field_ _ _          -> []
  H.FieldStructured n _ -> [(hdrName n, True, mempty & #type ?~ OA.OpenApiString)]
  H.Cookie{}            -> []
  H.Cookie'{}           -> []
  H.SetCookie{}         -> []
extractResHeaders (FMap _ c) = extractResHeaders c
extractResHeaders (LMap _ c) = extractResHeaders c
extractResHeaders (Apply cf cx) = extractResHeaders cf ++ extractResHeaders cx
extractResHeaders (Pure _) = []

extractContentType :: Codec (Headers ctx) i o -> Maybe ByteString
extractContentType (Lift (Field_ k v)) | k == "content-type" = Just v
extractContentType (FMap _ c) = extractContentType c
extractContentType (LMap _ c) = extractContentType c
extractContentType (Apply cf cx) = extractContentType cf <|> extractContentType cx
extractContentType _ = Nothing

bodySchemaOf :: forall (b :: Type -> Type -> Type) ctx a. IsoJson a => b ctx (IO a) -> OA.Schema
bodySchemaOf _ = toSchema (Proxy @a)

bodyDefsOf :: forall (b :: Type -> Type -> Type) ctx a. IsoJson a => b ctx (IO a) -> OA.Definitions OA.Schema
bodyDefsOf _ = execDeclare (declareSchemaRef (Proxy @a)) mempty

extractBodySchema :: Codec (Body ctx) i o -> Maybe OA.Schema
extractBodySchema (Lift body) = case body of
  Body.Json -> Just (bodySchemaOf body)
  _         -> Nothing
extractBodySchema (FMap _ c) = extractBodySchema c
extractBodySchema (LMap _ c) = extractBodySchema c
extractBodySchema (Apply cf cx) = extractBodySchema cf <|> extractBodySchema cx
extractBodySchema (Pure _) = Nothing

extractBodyDefs :: Codec (Body ctx) i o -> OA.Definitions OA.Schema
extractBodyDefs (Lift body) = case body of
  Body.Json -> bodyDefsOf body
  _         -> mempty

```

```

extractBodyDefs (FMap _ c) = extractBodyDefs c
extractBodyDefs (LMap _ c) = extractBodyDefs c
extractBodyDefs (Apply cf cx) = extractBodyDefs cf <> extractBodyDefs cx
extractBodyDefs (Pure _) = mempty

data ResInfo = ResInfo
  { resStatus      :: HTTP.Status
  , resMediaType   :: Maybe ByteString
  , resBodySchema  :: Maybe OA.Schema
  , resBodyDefs    :: OA.Definitions OA.Schema
  , resHdrNames    :: [(Text, Bool, OA.Schema)]
  }

resInfoOf :: ORes.Response IsoCodec s h b -> ResInfo
resInfoOf res =
  ResInfo
    { resStatus      = fromMaybe HTTP.status200 (Status.extractStatus res.status.isoCodec)
    , resMediaType   = extractContentType res.headers.isoCodec
    , resBodySchema  = extractBodySchema res.body.isoCodec
    , resBodyDefs    = extractBodyDefs res.body.isoCodec
    , resHdrNames    = extractResHeaders res.headers.isoCodec
    }

extractResInfos :: Cases responses => Responses IsoCodec responses -> [ResInfo]
extractResInfos (Responses cs) =
  map (getConst . traverseResponses @IsoCodec @IsoCodec (\c -> Const (resInfoOf c))) (NE.toList cs)

hdrName :: HTTP.HeaderName -> Text
hdrName = T.pack . BS8.unpack . CI.original

mkParam :: Text -> ParamLocation -> Bool -> Param
mkParam n loc req_ =
  mempty
    & #name .~ n
    & #in .~ loc
    & #required ?~ req_

mkParamWithSchema :: Text -> ParamLocation -> Bool -> OA.Schema -> Param
mkParamWithSchema n loc req_ sc =
  mempty
    & #name .~ n
    & #in .~ loc
    & #required ?~ req_
    & #schema ?~ Inline sc

mkArrayParam :: Text -> Bool -> Query.ArrayStyle -> Param
mkArrayParam n req_ style =
  mempty
    & #name .~ n
    & #in .~ ParamQuery
    & #required ?~ req_
    & #style ?~ st
    & #explode ?~ ex

```

```

    & #schema ?~ Inline arraySchema
where
  arraySchema =
    mempty
    & #type ?~ OA.OpenApiArray
    & #items ?~ OA.OpenApiItemsObject (Inline (mempty & #type ?~ OA.OpenApiString))
  (st, ex) = case style of
    Query.Exploded      -> (OA.StyleForm, True)
    Query.CommaDelimited -> (OA.StyleForm, False)
    Query.SpaceDelimited -> (OA.StyleSpaceDelimited, False)
    Query.PipeDelimited  -> (OA.StylePipeDelimited, False)

mkResResponse :: ResInfo -> Response
mkResResponse ri =
  mempty
  & #description .~ T.pack (show (HTTP.statusCode (resStatus ri)))
  & applyResBodySchema (resMediaType ri) (resBodySchema ri)
  & applyResHeaders (resHdrNames ri)

applyResBodySchema :: Maybe ByteString -> Maybe OA.Schema -> Response -> Response
applyResBodySchema _ Nothing r = r
applyResBodySchema mt (Just sc) r =
  r & #content .~ IHM.singleton (mediaKey mt) (mempty & #schema ?~ Inline sc)

applyResHeaders :: [(Text, Bool, OA.Schema)] -> Response -> Response
applyResHeaders [] r = r
applyResHeaders hs r =
  r & #headers .~ IHM.fromList
  [ (name, Inline (mempty & #required ?~ req_ & #schema ?~ Inline sc))
  | (name, req_, sc) <- hs
  ]

applyReqBodySchema :: Maybe ByteString -> Maybe OA.Schema -> Operation -> Operation
applyReqBodySchema _ Nothing op = op
applyReqBodySchema mt (Just sc) op =
  op & #requestBody ?~ Inline
  ( mempty & #content .~ IHM.singleton (mediaKey mt) (mempty & #schema ?~ Inline sc) )

mediaKey :: Maybe ByteString -> MediaType
mediaKey = maybe "application/json" (fromString . BS8.unpack)

setMethod :: HTTP.StdMethod -> Operation -> PathItem -> PathItem
setMethod HTTP.GET    op pi_ = pi_{_pathItemGet    = Just op}
setMethod HTTP.POST    op pi_ = pi_{_pathItemPost    = Just op}
setMethod HTTP.PUT     op pi_ = pi_{_pathItemPut     = Just op}
setMethod HTTP.DELETE  op pi_ = pi_{_pathItemDelete  = Just op}
setMethod _            op pi_ = pi_{_pathItemGet    = Just op}

endpointToOpenApi :: Cases responses => Contract (Signature method path query headers body responses) -> OpenApi
endpointToOpenApi
  ( Request
    { method = IsoCodec methodCodec
    , path   = IsoCodec pathCodec

```

```

    , query   = IsoCodec queryCodec
    , headers = IsoCodec headersCodec
    , body    = IsoCodec bodyCodec
  }
  :-> resAlt
) =
let
  stdMeth = fromMaybe HTTP.GET (Method.extractMethod methodCodec)
  pieces  = walkPath pathCodec []
  qParams = extractQueryParams queryCodec
  hParams = extractHeaderParams headersCodec
  resInfos = extractResInfos resAlt
  reqBody  =
    if stdMeth `notElem` [HTTP.GET, HTTP.HEAD]
    then extractBodySchema bodyCodec
    else Nothing
  allDefs =
    extractBodyDefs bodyCodec
    <> foldMap resBodyDefs resInfos
  op =
    mempty
    & #parameters .~ map Inline (pathOParams pieces ++ qParams ++ hParams)
    & #responses .~ OA.Responses Nothing
    ( IHM.fromList
      [ (HTTP.statusCode (resStatus ri), Inline (mkResResponse ri))
      | ri <- resInfos
      ]
    )
    & applyReqBodySchema (extractContentType headersCodec) reqBody
in
  mempty
  & #info % #title .~ "API"
  & #info % #version .~ "0.1.0"
  & #components % #schemas .~ allDefs
  & #paths .~ IHM.singleton (pathTemplate pieces) (setMethod stdMeth op mempty)

class GenericOAPI (epF :: Type -> Type) where
  gOpenApi :: epF () -> OpenApi

instance GenericOAPI epF => GenericOAPI (D1 dm epF) where
  gOpenApi (M1 ep) = gOpenApi @epF ep

instance GenericOAPI epF => GenericOAPI (C1 cm epF) where
  gOpenApi (M1 ep) = gOpenApi @epF ep

instance (GenericOAPI epL, GenericOAPI epR) => GenericOAPI (epL :+: epR) where
  gOpenApi (epL :+: epR) = gOpenApi @epL epL <> gOpenApi @epR epR

instance Cases responses => GenericOAPI (S1 sm (Rec0 (Contract (Signature method path query headers body responses)
  gOpenApi (M1 (K1 ep)) = endpointToOpenApi ep

openApi ::
  forall server.

```

```
( Generic (server Contract)
, GenericOAPI (Rep (server Contract))
) =>
server Contract ->
OpenApi
openApi = gOpenApi @(Rep (server Contract)) . from
```


Okapi.Codec

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Codec (
  Codec (..),
  IsoCodec (..),
  Value (..),
  ParseError (..),
  Result (..),
  ForRequest,
  ForResponse,
  StateOf,
  ParseErrorOf,
  PartOf,
  Parser,
  Printer,
  (=.),
  cost,
  parser,
  printer,
)
where

import Data.Kind (Type)
import Data.Profunctor

data ForRequest
data ForResponse

type Codec :: (Type -> Type) -> Type -> Type -> Type
data Codec t i o where
  FMap :: (o -> o') -> Codec t i o -> Codec t i o'
  LMap :: (i -> i') -> Codec t i' o -> Codec t i o
  Pure :: o -> Codec t i o
  Apply :: Codec t i (o -> o') -> Codec t i o -> Codec t i o'
  Lift :: t a -> Codec t a a

newtype IsoCodec (t :: Type -> Type) a = IsoCodec {isoCodec :: Codec t a a}

newtype Value (t :: Type -> Type) a = Value {value :: a}

newtype ParseError (t :: Type -> Type) a = ParseError { parseError :: Maybe (ParseErrorOf t) }

newtype Result (t :: Type -> Type) a = Result { result :: Either (ParseErrorOf t) a }

instance Functor (Codec t i) where
  fmap = FMap
```

```

instance Applicative (Codec t i) where
    pure = Pure
    (<*>) = Apply

instance Profunctor (Codec t) where
    rmap = FMap
    lmap = LMap

(=.) :: Profunctor p => (a -> b) -> p b c -> p a c
(=.) = lmap
infixr 5 =.

type family StateOf (t :: Type -> Type) :: Type
type family ParseErrorOf (t :: Type -> Type) :: Type

type family PartOf (t :: Type -> Type) :: Type

type Parser t a = StateOf t -> (Either (ParseErrorOf t) a, StateOf t)
type Printer t a = a -> StateOf t

cost :: Codec t i o -> Int
cost = \case
    FMap _ c -> cost c
    LMap _ c -> cost c
    Pure _ -> 0
    Apply c c' -> cost c + cost c'
    Lift _ -> 1

parser ::
    forall t i o.
    (forall a. t a -> Parser t a) ->
    Codec t i o ->
    Parser t o
parser alg = go
    where
        go :: forall i' o'. Codec t i' o' -> Parser t o'
        go (Pure x) s = (Right x, s)
        go (FMap f c) s = case go c s of
            (Left e, s') -> (Left e, s')
            (Right x, s') -> (Right (f x), s')
        go (LMap _ c) s = go c s
        go (Apply cf cx) s = case go cf s of
            (Left e, s1) -> (Left e, s1)
            (Right f, s1) -> case go cx s1 of
                (Left e, s2) -> (Left e, s2)
                (Right x, s2) -> (Right (f x), s2)
        go (Lift t) s = alg t s

printer ::
    forall t i o.
    (Monoid (StateOf t)) =>
    (forall a. t a -> Printer t a) ->
    Codec t i o ->

```

```
Printer t i
printer alg = go
  where
    go :: forall i' o'. Codec t i' o' -> Printer t i'
    go (Pure _) _ = mempty
    go (FMap _ c) i = go c i
    go (LMap f c) i = go c (f i)
    go (Apply cf cx) i = go cf i <> go cx i
    go (Lift t) i = alg t i
```

Okapi.Contract

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE QuantifiedConstraints #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Contract (
    Signature,
    Contract (..),
    parseRequest,
    parseRequestResult,
    printRequest,
    linkTo,
    parseResponse,
    parseResponseResult,
    parseResponses,
    printResponse,
) where

import Data.ByteString qualified as BS
import Data.ByteString.Lazy qualified as LBS
import Data.Functor.Identity (Identity (..))
import Data.IORef (newIORef, readIORef, writeIORef)
import Data.Kind (Type)
import Data.List.NonEmpty qualified as NE
import Data.Maybe (mapMaybe)
import Data.Text (Text)
import Data.Text qualified as T
import Data.Text.Encoding (decodeUtf8)
import Network.HTTP.Types qualified as HTTP
import Network.Wai qualified as Wai
import Okapi.Protocol.Request.Body qualified as Body
import Okapi.Codec (IsoCodec (..), ParseError (..), Result (..), Value (..))
import Okapi.Protocol.Headers qualified as Headers
import Okapi.Protocol.Request (Request)
import Okapi.Protocol.Request qualified as OkReq
import Okapi.Protocol.Request.Method qualified as Method
import Okapi.Protocol.Request.Path qualified as Path
import Okapi.Protocol.Request.Query qualified as Query
```

```

import Okapi.Protocol.Response
  ( Response
  , Cases (..)
  , Responses (Responses)
  , parseResponseResult
  , printOne
  , resultToParseError
  , resultToValue
  , traverseResponses
  , zipResponses
  )

data Signature
  (method    :: Type)
  (path      :: Type)
  (query     :: Type)
  (headers   :: Type)
  (body      :: Type)
  (responses :: ((Type -> Type) -> Type -> Type) -> Type)

data Contract sig where
  (:->) ::
    Request IsoCodec method path query headers body ->
    Responses IsoCodec responses ->
    Contract (Signature method path query headers body responses)

parseRequest ::
  Contract (Signature method path query headers body responses) ->
  Wai.Request ->
  IO (Either (Request ParseError method path query headers body) (Request Value method path query headers body))
parseRequest (req :-> _) waiReq = do
  let (mr, _) = Method.parse req.method.isoCodec (Wai.requestMethod waiReq)
      pr      = Path.parseExact req.path.isoCodec (Wai.pathInfo waiReq)
      (qr, _) = Query.parse req.query.isoCodec (Wai.queryString waiReq)
      (hr, _) = Headers.parse req.headers.isoCodec (Wai.requestHeaders waiReq)
  case (mr, pr, qr, hr) of
    (Right m, Right p, Right q, Right h) -> do
      bodyRaw <- Wai.strictRequestBody waiReq
      let (brE, _) = Body.parse req.body.isoCodec bodyRaw
      br <- traverse id brE
      pure $ case br of
        Right b -> Right $ OkReq.requestValue m p q h (pure b)
        Left e -> Left OkReq.Request
          { method = ParseError Nothing
          , path   = ParseError Nothing
          , query  = ParseError Nothing
          , headers = ParseError Nothing
          , body   = ParseError (Just e)
          }
    _ -> pure $ Left OkReq.Request
      { method = ParseError (either Just (const Nothing) mr)
      , path   = ParseError (either (Just . fst) (const Nothing) pr)
      , query  = ParseError (either Just (const Nothing) qr)

```

```

    , headers = ParseError (either Just (const Nothing) hr)
    , body    = ParseError Nothing
  }

parseRequestResult ::
  Contract (Signature method path query headers body responses) ->
  Wai.Request ->
  IO (Request Result method path query headers body)
parseRequestResult (req :-> _) waiReq = do
  bodyRaw <- Wai.strictRequestBody waiReq
  let (mr, _) = Method.parse req.method.isCodec (Wai.requestMethod waiReq)
      pr      = Path.parseExact req.path.isCodec (Wai.pathInfo waiReq)
      (qr, _) = Query.parse req.query.isCodec (Wai.queryString waiReq)
      (hr, _) = Headers.parse req.headers.isCodec (Wai.requestHeaders waiReq)
      (brE, _) = Body.parse req.body.isCodec bodyRaw
  pure OkReq.Request
    { method = Result mr
    , path   = Result (either (Left . fst) Right pr)
    , query  = Result qr
    , headers = Result hr
    , body   = Result brE
    }

printRequest ::
  Contract (Signature method path query headers body responses) ->
  Request Value method path query headers body ->
  IO Wai.Request
printRequest (req :-> _) rv = do
  bodyBytes <- Body.printM req.body.isCodec rv.body.value
  bodyRef   <- newIORef (LBS.toChunks bodyBytes)
  let streamBody = do
    chunks <- readIORef bodyRef
    case chunks of
      [] -> pure BS.empty
      (c:cs) -> writeIORef bodyRef cs >> pure c
  let baseReq = Wai.defaultRequest
    { Wai.requestMethod = Method.print req.method.isCodec rv.method.value
    , Wai.pathInfo      = Path.print req.path.isCodec rv.path.value
    , Wai.queryString   = Query.print req.query.isCodec rv.query.value
    , Wai.requestHeaders = Headers.print req.headers.isCodec rv.headers.value
    }
  pure (Wai.setRequestBodyChunks streamBody baseReq)

linkTo ::
  Contract (Signature method path query headers body responses) ->
  Request Value method path query headers body ->
  Text
linkTo (req :-> _) rv =
  let segs = Path.print req.path.isCodec rv.path.value
      query = Query.print req.query.isCodec rv.query.value
  in "/" <> T.intercalate "/" segs <> decodeUtf8 (HTTP.renderQuery True query)

parseResponse ::

```

```

    Response IsoCodec status headers body ->
    Wai.Response ->
    IO (Either (Response ParseError status headers body) (Response Value status headers body))
parseResponse res waiRes = do
    rr <- parseResponseResult res waiRes
    pure $ maybe (Left (resultToParseError rr)) Right (resultToValue rr)

parseResponses ::
    forall method path query headers body responses.
    Cases responses =>
    Contract (Signature method path query headers body responses) -> Wai.Response ->
    IO (Either (Responses ParseError responses) (responses Value))
parseResponses (_ :-> Responses cs) waiRes = do
    rs <- traverse parseBranch cs
    pure $ case mapMaybe toValue (NE.toList rs) of
        (v : _) -> Right v
        [] -> Left (Responses (fmap toErrors rs))
    where
        parseBranch :: responses IsoCodec -> IO (responses Result)
        parseBranch = traverseResponses @IsoCodec @Result (\codec -> parseResponseResult codec waiRes)
        toValue :: responses Result -> Maybe (responses Value)
        toValue = traverseResponses @Result @Value resultToValue
        toErrors :: responses Result -> responses ParseError
        toErrors = runIdentity . traverseResponses @Result @ParseError (Identity . resultToParseError)

printResponse ::
    forall method path query headers body responses.
    Cases responses =>
    Contract (Signature method path query headers body responses) ->
    responses Value ->
    IO Wai.Response
printResponse (_ :-> Responses cs) rv =
    case [io | c <- NE.toList cs, Just io <- [zipResponses @IsoCodec @Value printOne c rv]] of
        (io : _) -> io
        [] -> error "printResponse: no matching response constructor"

```

Okapi.Data

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE NoFieldSelectors #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE StandaloneKindSignatures #-}

module Okapi.Data
  ( Iso (..)
  , Info (..)
  , Data (..)

  , Prim (..)
  , numPrim, timePrim
  , intPrim, int16Prim, int32Prim, int64Prim, integerPrim
  , boolPrim, charPrim, floatPrim, doublePrim, scientificPrim, textPrim
  , dayPrim, localTimePrim, utcTimePrim, timeOfDayPrim, uuidPrim

  , int, int16, int32, int64, integer
  , bool, char, float, double, scientific, text
  , day, localTime, utcTime, timeOfDay, uuid
  ) where

import Data.Int (Int16, Int32, Int64)
import Data.Kind (Type)
import Data.Scientific (Scientific)
import Data.Text (Text)
import Data.Text qualified as T
import Data.Time (Day, LocalTime, TimeOfDay, UTCTime)
import Data.Time.Format (FormatTime, ParseTime, defaultTimeLocale, formatTime, parseTimeM)
import Data.UUID (UUID)
import Data.UUID qualified as UUID
import Okapi.Codec (ParseErrorOf, PartOf)
import Text.Read (readMaybe)

data Info = Info
  { typeName :: Text
  , format   :: Maybe Text
  }
  deriving (Eq, Show)

type Iso :: (Type -> Type) -> Type -> Type
data Iso t a = Iso
  { decode :: PartOf t -> Either (ParseErrorOf t) a
  , encode :: a -> PartOf t
  , info   :: Info
  }

class Data t a where
  iso :: Iso t a

data Prim a = Prim
```



```

    { primDecode :: Text -> Either Text a
    , primEncode :: a -> Text
    , primInfo   :: Info
    }

numPrim :: (Show a, Read a) => Text -> Maybe Text -> Prim a
numPrim ty fmt = Prim
    { primDecode = \t -> maybe (Left ("invalid " <> ty <> ": " <> t)) Right (readMaybe (T.unpack t))
    , primEncode = T.pack . show
    , primInfo   = Info ty fmt
    }

timePrim :: (FormatTime a, ParseTime a) => String -> Maybe Text -> Prim a
timePrim fmtStr fmt = Prim
    { primDecode = \t -> maybe (Left ("invalid " <> tn <> ": " <> t)) Right
        (parseTimeM True defaultTimeLocale fmtStr (T.unpack t))
    , primEncode = T.pack . formatTime defaultTimeLocale fmtStr
    , primInfo   = Info "string" fmt
    }
    where tn = maybe "time" id fmt

intPrim      :: Prim Int      ; intPrim      = numPrim "integer" Nothing
int16Prim    :: Prim Int16    ; int16Prim    = numPrim "integer" (Just "int32")
int32Prim    :: Prim Int32    ; int32Prim    = numPrim "integer" (Just "int32")
int64Prim    :: Prim Int64    ; int64Prim    = numPrim "integer" (Just "int64")
integerPrim  :: Prim Integer  ; integerPrim  = numPrim "integer" Nothing
floatPrim    :: Prim Float    ; floatPrim    = numPrim "number" (Just "float")
doublePrim   :: Prim Double   ; doublePrim   = numPrim "number" (Just "double")
scientificPrim :: Prim Scientific ; scientificPrim = numPrim "number" Nothing

textPrim :: Prim Text
textPrim = Prim Right id (Info "string" Nothing)

boolPrim :: Prim Bool
boolPrim = Prim dec (\b -> if b then "true" else "false") (Info "boolean" Nothing)
    where
        dec t = case T.toLower t of
            "true"  -> Right True
            "1"     -> Right True
            "false" -> Right False
            "0"     -> Right False
            _       -> Left ("invalid boolean: " <> t)

charPrim :: Prim Char
charPrim = Prim dec T.singleton (Info "string" Nothing)
    where
        dec t = case T.unpack t of
            [c] -> Right c
            _    -> Left ("invalid char: " <> t)

uuidPrim :: Prim UUID
uuidPrim = Prim dec UUID.toText (Info "string" (Just "uuid"))
    where

```

```

dec t = maybe (Left ("invalid uuid: " <> t)) Right (UUID.fromText t)

dayPrim      :: Prim Day      ; dayPrim      = timePrim "%Y-%m-%d" (Just "date")
localTimePrim :: Prim LocalTime ; localTimePrim = timePrim "%Y-%m-%dT%H:%M:%S" (Just "date-time")
utcTimePrim  :: Prim UTCTime  ; utcTimePrim  = timePrim "%Y-%m-%dT%H:%M:%SZ" (Just "date-time")
timeOfDayPrim :: Prim TimeOfDay ; timeOfDayPrim = timePrim "%H:%M:%S" (Just "time")

int          :: Data t Int      => Iso t Int      ; int          = iso
int16        :: Data t Int16    => Iso t Int16    ; int16        = iso
int32        :: Data t Int32    => Iso t Int32    ; int32        = iso
int64        :: Data t Int64    => Iso t Int64    ; int64        = iso
integer      :: Data t Integer  => Iso t Integer  ; integer      = iso
bool         :: Data t Bool     => Iso t Bool     ; bool         = iso
char         :: Data t Char     => Iso t Char     ; char         = iso
float        :: Data t Float    => Iso t Float    ; float        = iso
double       :: Data t Double   => Iso t Double   ; double       = iso
scientific   :: Data t Scientific => Iso t Scientific ; scientific   = iso
text         :: Data t Text     => Iso t Text     ; text         = iso
day          :: Data t Day      => Iso t Day      ; day          = iso
localTime    :: Data t LocalTime => Iso t LocalTime ; localTime    = iso
utcTime      :: Data t UTCTime  => Iso t UTCTime  ; utcTime      = iso
timeOfDay    :: Data t TimeOfDay => Iso t TimeOfDay ; timeOfDay    = iso
uuid         :: Data t UUID     => Iso t UUID     ; uuid         = iso

```

Okapi.Protocol.Body

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Body (
  Body (..),
  IsoJson,
  ParseError (..),
  parse,
  printM,
  raw,
  json,
  form,
  html,
  noContent,
) where

import Data.Aeson qualified as Aeson
import Data.ByteString.Lazy qualified as LBS
import Data.Kind (Type)
import Data.OpenApi (ToSchema)
import Lucid (Html, renderBS)
import Okapi.Codec (Codec (..), ForRequest, ForResponse, ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Web.FormUrlEncoded (FromForm, ToForm, urlDecodeAsForm, urlEncodeAsForm)

type IsoJson a = (Aeson.FromJSON a, Aeson.ToJSON a, ToSchema a)

type Body :: Type -> Type -> Type
data Body ctx a where
  Raw      :: Body ctx (IO LBS.ByteString)
  Json     :: IsoJson a => Body ctx (IO a)
  NoContent :: Body ctx (IO ())
  Form     :: (ToForm a, FromForm a) => Body ForRequest (IO a)
  Html     :: Body ForResponse (IO (Html ()))

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf (Body ctx) = LBS.ByteString
type instance ParseErrorOf (Body ctx) = ParseError

parse :: forall ctx i o. Codec (Body ctx) i o -> LBS.ByteString -> (Either ParseError o, LBS.ByteString)
parse = Codec.parser bodyAlg
  where
    bodyAlg :: forall a. Body ctx a -> LBS.ByteString -> (Either ParseError a, LBS.ByteString)
    bodyAlg Raw      bs = (Right (pure bs), LBS.empty)
    bodyAlg Json     bs = case Aeson.eitherDecode bs of
      Left _ -> (Left ParseError, bs)
      Right x -> (Right (pure x), LBS.empty)
```

```

bodyAlg Form      bs = case urlDecodeAsForm bs of
  Left _  -> (Left ParseError, bs)
  Right x -> (Right (pure x), LBS.empty)
bodyAlg Html      _ = (Left ParseError, LBS.empty)
bodyAlg NoContent _ = (Right (pure ()), LBS.empty)

printM :: forall ctx i o. Codec (Body ctx) i o -> i -> IO LBS.ByteString
printM = go
  where
    go :: forall i' o'. Codec (Body ctx) i' o' -> i' -> IO LBS.ByteString
    go (Pure _)      _      = pure mempty
    go (FMap _ c)    i      = go c i
    go (LMap f c)    i      = go c (f i)
    go (Apply cf cx) i      = liftA2 (< >) (go cf i) (go cx i)
    go (Lift Raw)    ioLbs = ioLbs
    go (Lift Json)   ioA  = Aeson.encode <$> ioA
    go (Lift Form)   ioA  = urlEncodeAsForm <$> ioA
    go (Lift Html)   ioH  = renderBS <$> ioH
    go (Lift NoContent) _ = pure mempty

raw :: Codec (Body ctx) (IO LBS.ByteString) (IO LBS.ByteString)
raw = Lift Raw

json :: IsoJson a => Codec (Body ctx) (IO a) (IO a)
json = Lift Json

form :: (ToForm a, FromForm a) => Codec (Body ForRequest) (IO a) (IO a)
form = Lift Form

html :: Codec (Body ForResponse) (IO (Html ())) (IO (Html ()))
html = Lift Html

noContent :: Codec (Body ctx) (IO ()) (IO ())
noContent = Lift NoContent

```

Okapi.Protocol.Headers

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Protocol.Headers (
    Headers (..),
    ParseError (..),
    MediaType (..),
    parse,
    print,
    raw,
    field,
    field',
    field_,
    fieldStructured,
    fieldBareItem,
    fieldItem,
    fieldList,
    fieldDictionary,
    mediaTypeBytes,
    contentType,
    cookie,
    cookie',
    setCookie,
    ConstF (..),
    GHeaders (..),
    headersCodec,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.CaseInsensitive qualified as CI
import Data.Int (Int16, Int32, Int64)
import Data.Kind (Type)
import Data.List (find, partition)
import Data.Proxy (Proxy (..))
import Data.Scientific (Scientific)
import Data.Text qualified as Text
import Data.Text.Encoding (decodeUtf8Lenient, encodeUtf8)
import Data.Time (Day, DiffTime, LocalTime, TimeOfDay, TimeZone, UTCTime, localTimeOfDay, timeZoneOffsetString, zoneName)
import Data.Time.Format (defaultTimeLocale, formatTime, parseTimeM)
import Data.UUID (UUID)
import GHC.Generics (C1, D1, Generic (..), K1 (..), M1 (..), Rec0, S1, Selector (..), (:*:) (..))
```

```

import GHC.TypeLits (KnownSymbol, Symbol, symbolVal)
import Network.HTTP.Types qualified as HTTP
import Text.Read (readMaybe)
import Web.Cookie qualified as WC
import Prelude hiding (print)

import Okapi.Codec (Codec (..), ForRequest, ForResponse, ParseErrorOf, PartOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data
  ( Data (..), Info (..), Iso (..), Prim (..)
  , boolPrim, charPrim, dayPrim, doublePrim, floatPrim, int16Prim, int32Prim
  , int64Prim, intPrim, integerPrim, localTimePrim, scientificPrim, textPrim
  , timeOfDayPrim, utcTimePrim, uuidPrim
  )
import Okapi.Protocol.Headers.Attributes (Attributes)
import Okapi.Protocol.Headers.Attributes qualified as Attr
import Okapi.Protocol.Headers.Cookies (Cookie)
import Okapi.Protocol.Headers.Structured (Structured)
import Okapi.Protocol.Headers.Structured qualified as Structured
import Okapi.Protocol.Headers.Structured.BareItem (BareItem)
import Okapi.Protocol.Headers.Structured.Item (Item, bareItem)
import Okapi.Protocol.Headers.Structured.List (List)
import Okapi.Protocol.Headers.Structured.Dictionary (Dictionary)

type Headers :: Type -> Type -> Type
data Headers ctx a where
  Raw          :: Headers ctx HTTP.RequestHeaders
  Field        :: HTTP.HeaderName -> Iso (Headers ctx) a -> Headers ctx a
  Field'       :: HTTP.HeaderName -> Iso (Headers ctx) a -> Headers ctx (Maybe a)
  Field_       :: HTTP.HeaderName -> ByteString -> Headers ctx ()
  FieldStructured :: HTTP.HeaderName -> Codec Structured a a -> Headers ctx a
  Cookie       :: ByteString -> Iso Cookie a -> Headers ForRequest a
  Cookie'      :: ByteString -> Iso Cookie a -> Headers ForRequest (Maybe a)
  SetCookie    :: ByteString -> Iso Cookie a -> Codec Attributes p p -> Headers ForResponse (a, p)

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf      (Headers ctx) = [HTTP.Header]
type instance ParseErrorOf (Headers ctx) = ParseError
type instance PartOf       (Headers ctx) = ByteString

parse :: forall ctx i o. Codec (Headers ctx) i o -> [HTTP.Header] -> (Either ParseError o, [HTTP.Header])
parse = Codec.parser headersAlg
  where
    headersAlg :: forall a. Headers ctx a -> HTTP.RequestHeaders -> (Either ParseError a, HTTP.RequestHeaders)
    headersAlg Raw hs = (Right hs, [])
    headersAlg (Field key vIso) hs =
      case partition \(k, _) -> k == key) hs of
        ([], _)      -> (Left ParseError, hs)

```

```

    ((_, v) : _, rest) -> case vIso.decode v of
      Left _ -> (Left ParseError, hs)
      Right x -> (Right x, rest)
headersAlg (Field' key vIso) hs =
  case partition (\(k, _) -> k == key) hs of
    ([], _) -> (Right Nothing, hs)
    ((_, v) : _, rest) -> case vIso.decode v of
      Left _ -> (Right Nothing, rest)
      Right x -> (Right (Just x), rest)
headersAlg (Field_ k v) hs =
  case lookup k hs of
    Just v' | v' == v -> (Right (), filter (\(k', _) -> k' /= k) hs)
    _ -> (Left ParseError, hs)
headersAlg (FieldStructured name c) hs =
  case partition (\(k, _) -> k == name) hs of
    ([], _) -> (Left ParseError, hs)
    ((_, v) : _, rest) ->
      case fst (Structured.parseStructured c v) of
        Left _ -> (Left ParseError, hs)
        Right x -> (Right x, rest)
headersAlg (Cookie name vIso) hs =
  case lookup name (reqCookiePairs hs) of
    Just v -> case vIso.decode v of
      Left _ -> (Left ParseError, hs)
      Right x -> (Right x, hs)
    Nothing -> (Left ParseError, hs)
headersAlg (Cookie' name vIso) hs =
  case lookup name (reqCookiePairs hs) of
    Just v -> (Right (either (const Nothing) Just (vIso.decode v)), hs)
    Nothing -> (Right Nothing, hs)
headersAlg (SetCookie name vIso attrsC) hs =
  case find (\(n, _, _) -> n == name) (tokenizeSetCookies hs) of
    Just (_, v, attrs) -> case vIso.decode v of
      Left _ -> (Left ParseError, hs)
      Right x -> case fst (Attr.parse attrsC attrs) of
        Left _ -> (Left ParseError, hs)
        Right p -> (Right (x, p), hs)
    Nothing -> (Left ParseError, hs)

reqCookiePairs :: [HTTP.Header] -> WC.Cookies
reqCookiePairs hs = WC.parseCookies (BS.intersperse ";" " [v | (n, v) <- hs, n == "cookie"]

tokenizeSetCookies :: [HTTP.Header] -> [(ByteString, ByteString, ByteString)]
tokenizeSetCookies hs = [tok v | (n, v) <- hs, n == "set-cookie"]
  where
    tok v =
      let (name, r1) = BS.break (== 61) v
          (val, attrs) = BS.break (== 59) (BS.drop 1 r1)
      in (name, val, attrs)

print :: forall ctx i o. Codec (Headers ctx) i o -> i -> [HTTP.Header]

```

```

print c i = coalesceCookies (Codec.printer headersPrinter c i)
  where
    headersPrinter :: forall a. Headers ctx a -> a -> HTTP.RequestHeaders
    headersPrinter Raw hs = hs
    headersPrinter (Field key vIso) x = [(key, vIso.encode x)]
    headersPrinter (Field' _ _) Nothing = []
    headersPrinter (Field' key vIso) (Just x) = [(key, vIso.encode x)]
    headersPrinter (Field_ k v) () = [(k, v)]
    headersPrinter (FieldStructured name c) a = [(name, Structured.printStructured c a)]
    headersPrinter (Cookie name vIso) x = [("cookie", name <> "=" <> vIso.encode x)]
    headersPrinter (Cookie' _ _) Nothing = []
    headersPrinter (Cookie' name vIso) (Just x) = [("cookie", name <> "=" <> vIso.encode x)]
    headersPrinter (SetCookie name vIso attrsC) (x, p) =
      [("set-cookie", name <> "=" <> vIso.encode x <> Attr.print attrsC p)]

coalesceCookies :: [HTTP.Header] -> [HTTP.Header]
coalesceCookies hs =
  let (cks, rest) = partition ((== "cookie") . fst) hs
  in rest ++ [("cookie", BS.intercalate "; " (map snd cks)) | not (null cks)]

data MediaType
  = JSON
  | HTML
  | PlainText
  | FormUrlEncoded
  | OctetStream
  | EventStream
  | Custom ByteString
  deriving (Eq, Show)

mediaTypeBytes :: MediaType -> ByteString
mediaTypeBytes JSON      = "application/json"
mediaTypeBytes HTML      = "text/html"
mediaTypeBytes PlainText = "text/plain"
mediaTypeBytes FormUrlEncoded = "application/x-www-form-urlencoded"
mediaTypeBytes OctetStream = "application/octet-stream"
mediaTypeBytes EventStream = "text/event-stream"
mediaTypeBytes (Custom bs) = bs

raw :: Codec (Headers ctx) HTTP.RequestHeaders HTTP.RequestHeaders
raw = Lift Raw

field :: HTTP.HeaderName -> Iso (Headers ctx) a -> Codec (Headers ctx) a a
field key vIso = Lift (Field key vIso)

field' :: HTTP.HeaderName -> Iso (Headers ctx) a -> Codec (Headers ctx) (Maybe a) (Maybe a)
field' key vIso = Lift (Field' key vIso)

field_ :: HTTP.HeaderName -> ByteString -> Codec (Headers ctx) h h -> Codec (Headers ctx) h h

```



```

field_ k v c = Apply (LMap (const ())) (FMap (const id) (Lift (Field_ k v)))) c

fieldStructured :: HTTP.HeaderName -> Codec Structured a a -> Codec (Headers ctx) a a
fieldStructured name c = Lift (FieldStructured name c)

fieldBareItem :: HTTP.HeaderName -> Iso BareItem a -> Codec (Headers ctx) a a
fieldBareItem name i = fieldStructured name (Structured.item (bareItem i))

fieldItem :: HTTP.HeaderName -> Codec Item a a -> Codec (Headers ctx) a a
fieldItem name c = fieldStructured name (Structured.item c)

fieldList :: HTTP.HeaderName -> Codec List a a -> Codec (Headers ctx) a a
fieldList name c = fieldStructured name (Structured.list c)

fieldDictionary :: HTTP.HeaderName -> Codec Dictionary a a -> Codec (Headers ctx) a a
fieldDictionary name c = fieldStructured name (Structured.dictionary c)

contentType :: MediaType -> Codec (Headers ctx) h h -> Codec (Headers ctx) h h
contentType mt = field_ "content-type" (mediaTypeBytes mt)

cookie :: ByteString -> Iso Cookie a -> Codec (Headers ForRequest) a a
cookie name vIso = Lift (Cookie name vIso)

cookie' :: ByteString -> Iso Cookie a -> Codec (Headers ForRequest) (Maybe a) (Maybe a)
cookie' name vIso = Lift (Cookie' name vIso)

setCookie :: ByteString -> Iso Cookie a -> Codec Attributes p p -> Codec (Headers ForResponse) (a, p) (a, p)
setCookie name vIso attrsC = Lift (SetCookie name vIso attrsC)

headerIso :: Prim a -> Iso (Headers ctx) a
headerIso (Prim dec enc nfo) =
  Iso (\bs -> either (const (Left ParseError)) Right (dec (decodeUtf8Lenient bs)))
    (encodeUtf8 . enc)
    nfo

instance Data (Headers ctx) Int      where iso = headerIso intPrim
instance Data (Headers ctx) Int16   where iso = headerIso int16Prim
instance Data (Headers ctx) Int32    where iso = headerIso int32Prim
instance Data (Headers ctx) Int64    where iso = headerIso int64Prim
instance Data (Headers ctx) Integer where iso = headerIso integerPrim
instance Data (Headers ctx) Float    where iso = headerIso floatPrim
instance Data (Headers ctx) Double   where iso = headerIso doublePrim
instance Data (Headers ctx) Scientific where iso = headerIso scientificPrim
instance Data (Headers ctx) Bool     where iso = headerIso boolPrim
instance Data (Headers ctx) Char     where iso = headerIso charPrim
instance Data (Headers ctx) Text.Text where iso = headerIso textPrim
instance Data (Headers ctx) Day      where iso = headerIso dayPrim
instance Data (Headers ctx) LocalTime where iso = headerIso localTimePrim
instance Data (Headers ctx) UTCTime  where iso = headerIso utcTimePrim
instance Data (Headers ctx) TimeOfDay where iso = headerIso timeOfDayPrim
instance Data (Headers ctx) UUID     where iso = headerIso uuidPrim

```

```

instance Data (Headers ctx) ByteString where
  iso = Iso Right id (Info "string" Nothing)

instance Data (Headers ctx) DiffTime where
  iso = headerIso (Prim dec enc (Info "number" Nothing))
  where
    enc = Text.pack . show . (realToFrac :: DiffTime -> Double)
    dec t = case (readMaybe (Text.unpack t) :: Maybe Double) of
      Just d -> Right (realToFrac d)
      Nothing -> Left ("invalid difftime: " <> t)

instance Data (Headers ctx) (TimeOfDay, TimeZone) where
  iso = headerIso (Prim dec enc (Info "string" (Just "time")))
  where
    enc (tod, tz) = Text.pack (formatTime defaultTimeLocale "%T" tod <> timeZoneOffsetString tz)
    dec t = case parseTimeM True defaultTimeLocale "%T%Z" (Text.unpack t) of
      Just zt -> Right (localTimeOfDay (zonedTimeToLocalTime zt), zonedTimeZone zt)
      Nothing -> Left ("invalid (TimeOfDay, TimeZone): " <> t)

data ConstF (val :: Symbol) = ConstF deriving (Eq, Show)

fieldToHeaderName :: String -> HTTP.HeaderName
fieldToHeaderName = CI.mk . encodeUtf8 . Text.pack . map (\c -> if c == '_' then '-' else c)

class GHeaders ctx (f :: Type -> Type) where
  gHeadersCodec :: Codec (Headers ctx) (f ()) (f ())

instance (GHeaders ctx f) => GHeaders ctx (D1 meta f) where
  gHeadersCodec = FMap M1 $ LMap unM1 gHeadersCodec

instance (GHeaders ctx f) => GHeaders ctx (C1 meta f) where
  gHeadersCodec = FMap M1 $ LMap unM1 gHeadersCodec

instance (GHeaders ctx f, GHeaders ctx g) => GHeaders ctx (f *: g) where
  gHeadersCodec =
    Apply
      (FMap (\l r -> l *: r) (LMap (\(l *: _) -> l) gHeadersCodec))
      (LMap (\(_ *: r) -> r) gHeadersCodec)

instance (Selector s, Data (Headers ctx) a) => GHeaders ctx (S1 s (Rec0 a)) where
  gHeadersCodec =
    let key = fieldToHeaderName (selName (undefined :: S1 s (Rec0 a) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Field key (iso @(Headers ctx) @a))

instance {-# OVERLAPPING #-} (Selector s, Data (Headers ctx) a) => GHeaders ctx (S1 s (Rec0 (Maybe a))) where
  gHeadersCodec =
    let key = fieldToHeaderName (selName (undefined :: S1 s (Rec0 (Maybe a) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Field' key (iso @(Headers ctx) @a))

instance {-# OVERLAPPING #-} (Selector s, KnownSymbol val) => GHeaders ctx (S1 s (Rec0 (ConstF val))) where
  gHeadersCodec =

```

```

let k = fieldToHeaderName (selName (undefined :: S1 s (Rec0 (ConstF val)) ()))
    v = encodeUtf8 (Text.pack (symbolVal (Proxy @val)))
in FMap (\() -> M1 (K1 ConstF)) $ LMap (const ()) $ Lift (Field_ k v)

headersCodec :: forall ctx a. (Generic a, GHeaders ctx (Rep a)) => Codec (Headers ctx) a a
headersCodec = FMap (to @a) $ LMap (from @a) (gHeadersCodec @ctx @(Rep a))

```

Okapi.Protocol.Headers.Attributes

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Attributes (
  Attributes,
  attribute,
  attribute',
  flag,
  flag',
  raw,
  maxAge,
  domain,
  path,
  secure,
  httpOnly,
  parse,
  print,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.ByteString.Char8 qualified as BS8
import Data.Char (toLower)
import Data.Kind (Type)
import Data.List (find)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data (Data (..), Iso (..))
import Okapi.Protocol.Headers.Cookies (Cookie, ParseError (..), cookieIso)
import Prelude hiding (print)

type Attributes :: Type -> Type
data Attributes a where
  Attr  :: ByteString -> Iso Cookie a -> Attributes a
  Attr' :: ByteString -> Iso Cookie a -> Attributes (Maybe a)
  Flag  :: ByteString -> Attributes ()
  Flag' :: ByteString -> Attributes Bool
  Raw   :: Attributes ByteString

type instance StateOf Attributes = ByteString
type instance ParseErrorOf Attributes = ParseError

attribute :: ByteString -> Iso Cookie a -> Codec Attributes a a
attribute key vIso = Lift (Attr key vIso)

attribute' :: ByteString -> Iso Cookie a -> Codec Attributes (Maybe a) (Maybe a)
attribute' key vIso = Lift (Attr' key vIso)
```

```

flag :: ByteString -> Codec Attributes () ()
flag = Lift . Flag

flag' :: ByteString -> Codec Attributes Bool Bool
flag' = Lift . Flag'

raw :: Codec Attributes ByteString ByteString
raw = Lift Raw

maxAge :: Codec Attributes Int Int
maxAge = attribute "Max-Age" (iso @Cookie @Int)

domain :: Codec Attributes ByteString ByteString
domain = attribute "Domain" (iso @Cookie @ByteString)

path :: Codec Attributes ByteString ByteString
path = attribute "Path" (iso @Cookie @ByteString)

secure :: Codec Attributes () ()
secure = flag "Secure"

httpOnly :: Codec Attributes () ()
httpOnly = flag "HttpOnly"

parse :: Codec Attributes i o -> ByteString -> (Either ParseError o, ByteString)
parse = Codec.parser alg
  where
    alg :: forall a. Attributes a -> ByteString -> (Either ParseError a, ByteString)
    alg (Attr key vIso) s = case look key s of
      Just (Just v) -> case vIso.decode v of
        Left _ -> (Left ParseError, s)
        Right x -> (Right x, s)
      _ -> (Left ParseError, s)
    alg (Attr' key vIso) s = case look key s of
      Just (Just v) -> (Right (either (const Nothing) Just (vIso.decode v)), s)
      _ -> (Right Nothing, s)
    alg (Flag key) s = case look key s of
      Just _ -> (Right (), s)
      Nothing -> (Left ParseError, s)
    alg (Flag' key) s = (Right (maybe False (const True) (look key s)), s)
    alg Raw s = (Right s, s)

print :: Codec Attributes i o -> i -> ByteString
print = Codec.printer pr
  where
    pr :: forall a. Attributes a -> a -> ByteString
    pr (Attr key vIso) v      = ";" < key < "=" < vIso.encode v
    pr (Attr' key vIso) (Just v) = ";" < key < "=" < vIso.encode v
    pr (Attr' _ _)      Nothing = ""
    pr (Flag key)      ()      = ";" < key
    pr (Flag' key)     True     = ";" < key
    pr (Flag' _)       False    = ""
    pr Raw             bs       = bs

```

```

look :: ByteString -> ByteString -> Maybe (Maybe ByteString)
look key s = fmap snd (find ((== lc key) . fst) entries)
  where
    entries = [ breakEq (strip a) | a <- BS.split 59 s, not (BS.null (strip a)) ]
    breakEq a = case BS.elemIndex 61 a of
      Nothing -> (lc (strip a), Nothing)
      Just i   -> (lc (strip (BS.take i a)), Just (strip (BS.drop (i + 1) a)))

lc :: ByteString -> ByteString
lc = BS8.map toLower

strip :: ByteString -> ByteString
strip = BS.dropWhileEnd isSp . BS.dropWhile isSp
  where isSp w = w == 32 || w == 9

```

Okapi.Protocol.Headers.Cookies

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Cookies (
    Cookie,
    ParseError (..),
    cookieIso,
) where

import Data.ByteString (ByteString)
import Data.Int (Int16, Int32, Int64)
import Data.Kind (Type)
import Data.Scientific (Scientific)
import Data.Text (Text)
import Data.Text.Encoding (decodeUtf8Lenient, encodeUtf8)
import Data.Time (Day, LocalTime, TimeOfDay, UTCTime)
import Data.UUID (UUID)
import Okapi.Codec (ParseErrorOf, PartOf)
import Okapi.Data
    ( Data (..), Info (..), Iso (..), Prim (..)
    , boolPrim, charPrim, dayPrim, doublePrim, floatPrim
    , int16Prim, int32Prim, int64Prim, intPrim, integerPrim
    , localTimePrim, scientificPrim, textPrim
    , timeOfDayPrim, utcTimePrim, uuidPrim
    )

type Cookie :: Type -> Type
data Cookie a

data ParseError = ParseError deriving (Eq, Show)

type instance PartOf Cookie = ByteString
type instance ParseErrorOf Cookie = ParseError

cookieIso :: Prim a -> Iso Cookie a
cookieIso (Prim dec enc nfo) =
    Iso (\bs -> either (const (Left ParseError)) Right (dec (decodeUtf8Lenient bs)))
        (encodeUtf8 . enc)
        nfo

instance Data Cookie Int      where iso = cookieIso intPrim
instance Data Cookie Int16   where iso = cookieIso int16Prim
instance Data Cookie Int32   where iso = cookieIso int32Prim
instance Data Cookie Int64   where iso = cookieIso int64Prim
instance Data Cookie Integer where iso = cookieIso integerPrim
instance Data Cookie Float   where iso = cookieIso floatPrim
instance Data Cookie Double  where iso = cookieIso doublePrim
instance Data Cookie Scientific where iso = cookieIso scientificPrim
instance Data Cookie Bool    where iso = cookieIso boolPrim
```

```
instance Data Cookie Char      where iso = cookieIso charPrim
instance Data Cookie Text     where iso = cookieIso textPrim
instance Data Cookie Day      where iso = cookieIso dayPrim
instance Data Cookie LocalTime where iso = cookieIso localTimePrim
instance Data Cookie UTCTime  where iso = cookieIso utcTimePrim
instance Data Cookie TimeOfDay where iso = cookieIso timeOfDayPrim
instance Data Cookie UUID     where iso = cookieIso uuidPrim
instance Data Cookie ByteString where iso = Iso Right id (Info "string" Nothing)
```


Okapi.Protocol.Headers.Structured

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured (
  Structured,
  ParseError (..),
  item,
  list,
  dictionary,
  parseStructured,
  printStructured,
) where

import Data.ByteString (ByteString)
import Data.Kind (Type)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Protocol.Headers.Structured.Dictionary (Dictionary, parseDictionary, printDictionary)
import Okapi.Protocol.Headers.Structured.Item (Item, parseItem, printItem)
import Okapi.Protocol.Headers.Structured.Lexer (ParseError (..), strip, stripLead)
import Okapi.Protocol.Headers.Structured.List (List, parseList, printList)

type Structured :: Type -> Type
data Structured a where
  SItem :: Codec Item a a -> Structured a
  SList :: Codec List a a -> Structured a
  SDict :: Codec Dictionary a a -> Structured a

type instance StateOf Structured = ByteString
type instance ParseErrorOf Structured = ParseError

item :: Codec Item a a -> Codec Structured a a
item = Lift . SItem

list :: Codec List a a -> Codec Structured a a
list = Lift . SList

dictionary :: Codec Dictionary a a -> Codec Structured a a
dictionary = Lift . SDict

parseStructured :: Codec Structured i o -> ByteString -> (Either ParseError o, ByteString)
parseStructured = Codec.parser alg
  where
    alg :: forall a. Structured a -> ByteString -> (Either ParseError a, ByteString)
    alg (SItem c) s = (fst (parseItem c (strip s)), s)
    alg (SList c) s = (fst (parseList c (strip s)), s)
    alg (SDict c) s = (fst (parseDictionary c (strip s)), s)

printStructured :: Codec Structured i o -> i -> ByteString
```

```
printStructured = Codec.printer pr
  where
    pr :: forall a. Structured a -> a -> ByteString
    pr (SItem c) a = printItem c a
    pr (SList c) a = stripLead (printList c a)
    pr (SDict c) a = stripLead (printDictionary c a)
```

Okapi.Protocol.Headers.Structured.BareItem

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured.BareItem (
    BareItem,
    renderInner,
    parseInnerToList,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.ByteString.Char8 qualified as BS8
import Data.Scientific (FPFormat (Fixed), Scientific, formatScientific, toRealFloat)
import Data.Text (Text)
import Data.Text.Encoding (decodeUtf8Lenient, encodeUtf8)
import Data.Time.Clock (UTCTime)
import Data.Time.Clock.POSIX (posixSecondsToUTCTime, utcTimeToPOSIXSeconds)
import Okapi.Codec (ParseErrorOf, PartOf)
import Okapi.Data (Data (..), Info (..), Iso (..))
import Okapi.Protocol.Headers.Structured.Lexer (ParseError (..), eParse, parseInnerBare, strip)

data BareItem a

type instance PartOf BareItem = ByteString
type instance ParseErrorOf BareItem = ParseError

bareIso :: (ByteString -> Either Text a) -> (a -> ByteString) -> Info -> Iso BareItem a
bareIso dec enc nfo = Iso (eParse . dec) enc nfo

decInteger :: ByteString -> Either Text Integer
decInteger bs = case BS8.readInteger bs of
    Just (n, r) | BS.null r -> Right n
    _ -> Left ("invalid sf-integer: " <> decodeUtf8Lenient bs)

instance Data BareItem Integer where
    iso = bareIso decInteger (BS8.pack . show) (Info "integer" Nothing)

instance Data BareItem Int where
    iso = bareIso (fmap fromInteger . decInteger) (BS8.pack . show) (Info "integer" Nothing)

instance Data BareItem Bool where
    iso = bareIso dec enc (Info "boolean" Nothing)
    where
        enc True = "?1"
        enc False = "?0"
        dec "?1" = Right True
        dec "?0" = Right False
        dec bs = Left ("invalid sf-boolean: " <> decodeUtf8Lenient bs)
```

```

escapeSfString :: ByteString -> ByteString
escapeSfString = BS.concatMap esc
  where
    esc 34 = "\\\""
    esc 92 = "\\\""
    esc w  = BS.singleton w

parseSfString :: ByteString -> Either Text Text
parseSfString bs = case BS.uncons bs of
  Just (34, rest) -> go rest mempty
  _ -> Left ("invalid sf-string (no opening quote): " <> decodeUtf8Lenient bs)
  where
    go s acc = case BS.uncons s of
      Nothing -> Left "invalid sf-string (unterminated)"
      Just (34, r)
        | BS.null r -> Right (decodeUtf8Lenient acc)
        | otherwise -> Left "invalid sf-string (trailing bytes after close)"
      Just (92, r) -> case BS.uncons r of
        Just (c, r') | c == 34 || c == 92 -> go r' (acc <> BS.singleton c)
        _ -> Left "invalid sf-string (bad escape)"
      Just (c, r) -> go r (acc <> BS.singleton c)

instance Data BareItem Text where
  iso = bareIso parseSfString (\t -> "\"" <> escapeSfString (encodeUtf8 t) <> "\"") (Info "string" Nothing)

instance Data BareItem ByteString where
  iso = bareIso Right id (Info "string" Nothing)

renderSfDecimal :: Scientific -> ByteString
renderSfDecimal = BS8.pack . formatScientific Fixed (Just 3)

parseSfDecimal :: ByteString -> Either Text Scientific
parseSfDecimal bs = case BS8.readInt bs of
  _ -> case reads (BS8.unpack bs) of
    [(s, "")] -> Right s
    _ -> Left ("invalid sf-decimal: " <> decodeUtf8Lenient bs)

instance Data BareItem Scientific where
  iso = bareIso parseSfDecimal renderSfDecimal (Info "number" Nothing)

instance Data BareItem Double where
  iso = bareIso (fmap toRealFloat . parseSfDecimal) (renderSfDecimal . realToFrac) (Info "number" (Just "double"))

instance Data BareItem Float where
  iso = bareIso (fmap toRealFloat . parseSfDecimal) (renderSfDecimal . realToFrac) (Info "number" (Just "float"))

instance Data BareItem UTCTime where
  iso = bareIso dec enc (Info "string" (Just "date-time"))
  where
    enc t = BS8.cons '@' (BS8.pack (show (truncate (utcTimeToPOSIXSeconds t) :: Integer)))
    dec bs = case BS8.uncons bs of
      Just ('@', rest) -> case BS8.readInteger rest of
        Just (n, r) | BS.null r -> Right (posixSecondsToUTCTime (fromInteger n))

```

```

    _ -> Left ("invalid sf-date: " <> decodeUtf8Lenient bs)
    _ -> Left ("invalid sf-date (expected '@'): " <> decodeUtf8Lenient bs)

renderInner :: Iso BareItem a -> [a] -> ByteString
renderInner vIso xs = "(" <> BS.intercalate " " (map vIso.encode xs) <> ")"

parseInnerToList :: Iso BareItem a -> ByteString -> Either ParseError [a]
parseInnerToList vIso v = case parseInnerBare (strip v) of
    Just xs -> traverse vIso.decode xs
    Nothing -> Left ParseError

```

Okapi.Protocol.Headers.Structured.Dictionary

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured.Dictionary (
    Dictionary,
    member,
    member',
    list,
    list',
    raw,
    parseDictionary,
    printDictionary,
) where

import Data.ByteString (ByteString)
import Data.Kind (Type)
import Data.List (find)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data (Iso)
import Okapi.Protocol.Headers.Structured.BareItem (BareItem, parseInnerToList, renderInner)
import Okapi.Protocol.Headers.Structured.Item (Item, parseItem, printItem)
import Okapi.Protocol.Headers.Structured.Lexer (Key, ParseError (..), memberEntries)

type Dictionary :: Type -> Type
data Dictionary a where
    Member    :: Key -> Codec Item a a -> Dictionary a
    Member'   :: Key -> Codec Item a a -> Dictionary (Maybe a)
    List      :: Key -> Iso BareItem a -> Dictionary [a]
    List'     :: Key -> Iso BareItem a -> Dictionary (Maybe [a])
    Raw       :: Dictionary ByteString

type instance StateOf Dictionary = ByteString
type instance ParseErrorOf Dictionary = ParseError

member :: Key -> Codec Item a a -> Codec Dictionary a a
member k c = Lift (Member k c)

member' :: Key -> Codec Item a a -> Codec Dictionary (Maybe a) (Maybe a)
member' k c = Lift (Member' k c)

list :: Key -> Iso BareItem a -> Codec Dictionary [a] [a]
list k vIso = Lift (List k vIso)

list' :: Key -> Iso BareItem a -> Codec Dictionary (Maybe [a]) (Maybe [a])
list' k vIso = Lift (List' k vIso)

raw :: Codec Dictionary ByteString ByteString
raw = Lift Raw
```

```

parseDictionary :: Codec Dictionary i o -> ByteString -> (Either ParseError o, ByteString)
parseDictionary = Codec.parser alg
  where
    alg :: forall a. Dictionary a -> ByteString -> (Either ParseError a, ByteString)
    alg t s = case t of
      Member key c -> case look s key of
        Just (Just v) -> (fst (parseItem c v), s)
        Just Nothing -> (fst (parseItem c "?1"), s)
        Nothing -> (Left ParseError, s)
      Member' key c -> case look s key of
        Just (Just v) -> (fmap Just (fst (parseItem c v)), s)
        _ -> (Right Nothing, s)
      List key vIso -> case look s key of
        Just (Just v) -> (parseInnerToList vIso v, s)
        _ -> (Left ParseError, s)
      List' key vIso -> case look s key of
        Just (Just v) -> (fmap Just (parseInnerToList vIso v), s)
        _ -> (Right Nothing, s)
      Raw -> (Right s, s)
    look s k = fmap snd (find ((== k) . fst) (memberEntries s))

printDictionary :: Codec Dictionary i o -> i -> ByteString
printDictionary = Codec.printer pr
  where
    pr :: forall a. Dictionary a -> a -> ByteString
    pr (Member key c) a = ", " <> key <> "=" <> printItem c a
    pr (Member' key c) (Just a) = ", " <> key <> "=" <> printItem c a
    pr (Member' _ _) Nothing = ""
    pr (List key vIso) xs = ", " <> key <> "=" <> renderInner vIso xs
    pr (List' key vIso) (Just xs) = ", " <> key <> "=" <> renderInner vIso xs
    pr (List' _ _) Nothing = ""
    pr Raw bs = bs

```

Okapi.Protocol.Headers.Structured.Item

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured.Item (
  Item,
  bareItem,
  bareItemEq,
  item,
  params,
  raw,
  parseItem,
  printItem,
) where

import Data.ByteString (ByteString)
import Data.Kind (Type)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf, (=..))
import Okapi.Codec qualified as Codec
import Okapi.Data (Iso (..))
import Okapi.Protocol.Headers.Structured.BareItem (BareItem)
import Okapi.Protocol.Headers.Structured.Lexer (ParseError (..), firstAndRest, strip)
import Okapi.Protocol.Headers.Structured.Parameters (Parameters, parseParameters, printParameters)

type Item :: Type -> Type
data Item a where
  Bare    :: Iso BareItem a -> Item a
  BareEq  :: ByteString -> Item ()
  Params  :: Codec Parameters p p -> Item p
  Raw     :: Item ByteString

type instance StateOf Item = ByteString
type instance ParseErrorOf Item = ParseError

bareItem :: Iso BareItem a -> Codec Item a a
bareItem = Lift . Bare

bareItemEq :: ByteString -> Codec Item () ()
bareItemEq = Lift . BareEq

item :: Iso BareItem a -> Codec Parameters p p -> Codec Item (a, p) (a, p)
item vIso c = (,) <$> (fst =. Lift (Bare vIso)) <*> (snd =. Lift (Params c))

params :: Codec Parameters p p -> Codec Item p p
params = Lift . Params

raw :: Codec Item ByteString ByteString
raw = Lift Raw
```



```

parseItem :: Codec Item i o -> ByteString -> (Either ParseError o, ByteString)
parseItem = Codec.parser alg
  where
    alg :: forall a. Item a -> ByteString -> (Either ParseError a, ByteString)
    alg t s = case t of
      Bare vIso -> (vIso.decode (strip bare), s)
      BareEq c  -> (if strip bare == c then Right () else Left ParseError, s)
      Params c  -> (fst (parseParameters c ps), s)
      Raw       -> (Right s, s)
    where
      (bare, ps) = firstAndRest 59 s

printItem :: Codec Item i o -> i -> ByteString
printItem = Codec.printer pr
  where
    pr :: forall a. Item a -> a -> ByteString
    pr (Bare vIso) v = vIso.encode v
    pr (BareEq c)   () = c
    pr (Params c)   p = printParameters c p
    pr Raw          bs = bs

```

Okapi.Protocol.Headers.Structured.Lexer

```
{-# LANGUAGE OverloadedStrings #-}

module Okapi.Protocol.Headers.Structured.Lexer (
    ParseError (..),
    Key,
    eParse,
    strip,
    stripLead,
    firstTop,
    splitTop,
    firstAndRest,
    breakKey,
    paramEntries,
    memberEntries,
    parseInnerBare,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.Maybe (fromMaybe)
import Data.Word (Word8)

type Key = ByteString

data ParseError = ParseError deriving (Eq, Show)

eParse :: Either e a -> Either ParseError a
eParse = either (const (Left ParseError)) Right

paramEntries :: ByteString -> [(ByteString, Maybe ByteString)]
paramEntries = map breakKey . filter (not . BS.null) . map strip . splitTop 59

memberEntries :: ByteString -> [(ByteString, Maybe ByteString)]
memberEntries = map breakKey . filter (not . BS.null) . map strip . splitTop 44

breakKey :: ByteString -> (ByteString, Maybe ByteString)
breakKey bs = case firstTop 61 bs of
    Nothing -> (strip bs, Nothing)
    Just i -> (strip (BS.take i bs), Just (strip (BS.drop (i + 1) bs)))

parseInnerBare :: ByteString -> Maybe [ByteString]
parseInnerBare m = case BS.uncons (strip m) of
    Just (40, rest) -> case firstTop 41 rest of
        Just i -> Just (filter (not . BS.null) (map strip (splitTop 32 (BS.take i rest))))
        Nothing -> Nothing
    _ -> Nothing

firstTop :: Word8 -> ByteString -> Maybe Int
firstTop sep bs = go False (0 :: Int) 0
    where
        n = BS.length bs
        go inQ depth i
```

```

| i >= n = Nothing
| otherwise =
    let w = BS.index bs i
    in if w == 34 then go (not inQ) depth (i + 1)
       else if inQ then go inQ depth (i + 1)
       else if w == 40 then go inQ (depth + 1) (i + 1)
       else if w == 41 then go inQ (max 0 (depth - 1)) (i + 1)
       else if w == sep && depth == 0 then Just i
       else go inQ depth (i + 1)

splitTop :: Word8 -> ByteString -> [ByteString]
splitTop sep bs = case firstTop sep bs of
    Nothing -> [bs]
    Just i -> BS.take i bs : splitTop sep (BS.drop (i + 1) bs)

firstAndRest :: Word8 -> ByteString -> (ByteString, ByteString)
firstAndRest sep bs = case firstTop sep bs of
    Nothing -> (bs, BS.empty)
    Just i -> (BS.take i bs, BS.drop (i + 1) bs)

strip :: ByteString -> ByteString
strip = BS.dropWhileEnd isSp . BS.dropWhile isSp
    where isSp w = w == 32 || w == 9

stripLead :: ByteString -> ByteString
stripLead bs = fromMaybe bs (BS.stripPrefix " " bs)

```

Okapi.Protocol.Headers.Structured.List

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured.List (
    List,
    item,
    innerList,
    items,
    raw,
    parseList,
    printList,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.Kind (Type)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data (Iso)
import Okapi.Protocol.Headers.Structured.BareItem (BareItem, parseInnerToList, renderInner)
import Okapi.Protocol.Headers.Structured.Item (Item, parseItem, printItem)
import Okapi.Protocol.Headers.Structured.Lexer
    (ParseError (..), firstAndRest, splitTop, strip)

type List :: Type -> Type
data List a where
    ListItem    :: Codec Item a a -> List a
    InnerList   :: Iso BareItem a -> List [a]
    Items       :: Codec Item a a -> List [a]
    Raw         :: List ByteString

type instance StateOf List = ByteString
type instance ParseErrorOf List = ParseError

item :: Codec Item a a -> Codec List a a
item = Lift . ListItem

innerList :: Iso BareItem a -> Codec List [a] [a]
innerList = Lift . InnerList

items :: Codec Item a a -> Codec List [a] [a]
items = Lift . Items

raw :: Codec List ByteString ByteString
raw = Lift Raw

parseList :: Codec List i o -> ByteString -> (Either ParseError o, ByteString)
parseList = Codec.parser alg
    where
```

```

alg :: forall a. List a -> ByteString -> (Either ParseError a, ByteString)
alg t s = case t of
  ListItem c      -> let (m, rest) = firstAndRest 44 s
                      in (fst (parseItem c (strip m)), rest)
  InnerList vIso -> let (m, rest) = firstAndRest 44 s
                      in (parseInnerToList vIso (strip m), rest)
  Items c         -> (traverse (\m -> fst (parseItem c m)) (members s), BS.empty)
  Raw             -> (Right s, BS.empty)
members bs = filter (not . BS.null) (map strip (splitTop 44 bs))

printList :: Codec List i o -> i -> ByteString
printList = Codec.printer pr
  where
    pr :: forall a. List a -> a -> ByteString
    pr (ListItem c) a = ", " <> printItem c a
    pr (InnerList vIso) xs = ", " <> renderInner vIso xs
    pr (Items c) xs = BS.concat [", " <> printItem c x | x <- xs]
    pr Raw bs = bs

```

Okapi.Protocol.Headers.Structured.Parameters

```
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Headers.Structured.Parameters (
  Parameters,
  param,
  param',
  param_,
  flag,
  flag',
  raw,
  parseParameters,
  printParameters,
) where

import Data.ByteString (ByteString)
import Data.Kind (Type)
import Data.List (find)
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data (Iso (..))
import Okapi.Protocol.Headers.Structured.BareItem (BareItem)
import Okapi.Protocol.Headers.Structured.Lexer (Key, ParseError (..), paramEntries)

type Parameters :: Type -> Type
data Parameters a where
  Param  :: Key -> Iso BareItem a -> Parameters a
  Param' :: Key -> Iso BareItem a -> Parameters (Maybe a)
  Param_ :: Key -> Iso BareItem a -> a -> Parameters ()
  Flag   :: Key -> Parameters ()
  Flag'  :: Key -> Parameters Bool
  Raw    :: Parameters ByteString

type instance StateOf Parameters = ByteString
type instance ParseErrorOf Parameters = ParseError

param :: Key -> Iso BareItem a -> Codec Parameters a a
param k vIso = Lift (Param k vIso)

param' :: Key -> Iso BareItem a -> Codec Parameters (Maybe a) (Maybe a)
param' k vIso = Lift (Param' k vIso)

param_ :: Key -> Iso BareItem a -> a -> Codec Parameters () ()
param_ k vIso x = Lift (Param_ k vIso x)

flag :: Key -> Codec Parameters () ()
flag = Lift . Flag
```

```

flag' :: Key -> Codec Parameters Bool Bool
flag' = Lift . Flag'

raw :: Codec Parameters ByteString ByteString
raw = Lift Raw

parseParameters :: Codec Parameters i o -> ByteString -> (Either ParseError o, ByteString)
parseParameters = Codec.parser alg
  where
    alg :: forall a. Parameters a -> ByteString -> (Either ParseError a, ByteString)
    alg t s = case t of
      Param key vIso -> case look s key of
        Just (Just v) -> (vIso.decode v, s)
        _              -> (Left ParseError, s)
      Param' key vIso -> case look s key of
        Just (Just v) -> (Right (either (const Nothing) Just (vIso.decode v)), s)
        _              -> (Right Nothing, s)
      Param_ key vIso x -> case look s key of
        Just (Just v) | v == vIso.encode x -> (Right (), s)
        _                                  -> (Left ParseError, s)
      Flag key -> case look s key of
        Just Nothing      -> (Right (), s)
        Just (Just "?1") -> (Right (), s)
        _                 -> (Left ParseError, s)
      Flag' key -> case look s key of
        Just Nothing      -> (Right True, s)
        Just (Just "?1") -> (Right True, s)
        _                 -> (Right False, s)
      Raw -> (Right s, s)
    look s k = fmap snd (find ((== k) . fst) (paramEntries s))

printParameters :: Codec Parameters i o -> i -> ByteString
printParameters = Codec.printer pr
  where
    pr :: forall a. Parameters a -> a -> ByteString
    pr (Param key vIso) v      = ";" <> key <> "=" <> vIso.encode v
    pr (Param' key vIso) (Just v) = ";" <> key <> "=" <> vIso.encode v
    pr (Param' _ _)      Nothing = ""
    pr (Param_ key vIso x) ()    = ";" <> key <> "=" <> vIso.encode x
    pr (Flag key)          ()    = ";" <> key
    pr (Flag' key)         True  = ";" <> key
    pr (Flag' _)           False = ""
    pr Raw                 bs    = bs

```

Okapi.Protocol.Request

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE NoFieldSelectors #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Protocol.Request where

import Data.ByteString.Lazy qualified as LBS
import Data.Function ((&))
import Data.Kind (Type)
import Data.Text qualified as Text
import GHC.Generics (Generic, Rep)
import Network.HTTP.Types qualified as HTTP
import Okapi.Codec (Codec, ForRequest, IsoCodec (..), Value (..))
import Okapi.Protocol.Body (Body)
import Okapi.Protocol.Body qualified as Body
import Okapi.Protocol.Headers (Headers, GHeaders, headersCodec)
import Okapi.Protocol.Headers qualified as Headers
import Okapi.Protocol.Request.Method (KnownMethod (..), Method, GET, POST, DELETE)
import Okapi.Protocol.Request.Method qualified as Method
import Okapi.Protocol.Request.Path (Path, GPath)
import Okapi.Protocol.Request.Path qualified as Path
import Okapi.Protocol.Request.Query (Query, GQuery)
import Okapi.Protocol.Request.Query qualified as Query

data Request (mode :: (Type -> Type) -> Type -> Type) method path query headers body = Request
  { method   :: mode Method method
  , path     :: mode Path path
  , query    :: mode Query query
  , headers  :: mode (Headers ForRequest) headers
  , body     :: mode (Body ForRequest) (IO body)
  }

requestValue :: method -> path -> query -> headers -> IO body -> Request Value method path query headers body
requestValue m p q h b = Request
  { method   = Value m
  , path     = Value p
  , query    = Value q
  , headers  = Value h
  , body     = Value b
  }

request :: Request IsoCodec HTTP.Method [Text.Text] HTTP.Query [HTTP.Header] LBS.ByteString
request = Request
  { method   = IsoCodec Method.raw
  , path     = IsoCodec Path.raw
  , query    = IsoCodec Query.raw
  , headers  = IsoCodec Headers.raw
```



```

    , body    = IsoCodec Body.raw
  }

methodGET :: Request IsoCodec GET [Text.Text] HTTP.Query [HTTP.Header] LBS.ByteString
methodGET = request & method GET

methodPOST :: Request IsoCodec POST [Text.Text] HTTP.Query [HTTP.Header] LBS.ByteString
methodPOST = request & method POST

methodDELETE :: Request IsoCodec DELETE [Text.Text] HTTP.Query [HTTP.Header] LBS.ByteString
methodDELETE = request & method DELETE

method ::
  KnownMethod meth ->
  Request IsoCodec HTTP.Method path query headers body ->
  Request IsoCodec (KnownMethod meth) path query headers body
method km r = r { method = IsoCodec (Method.method km) }

path ::
  Codec Path path path ->
  Request IsoCodec meth [Text.Text] query headers body ->
  Request IsoCodec meth path query headers body
path c r = r { path = IsoCodec c }

query ::
  Codec Query query query ->
  Request IsoCodec meth path HTTP.Query headers body ->
  Request IsoCodec meth path query headers body
query c r = r { query = IsoCodec c }

headers ::
  Codec (Headers ForRequest) h h ->
  Request IsoCodec meth path query [HTTP.Header] bdy ->
  Request IsoCodec meth path query h bdy
headers c r = r { headers = IsoCodec c }

body ::
  Codec (Body ForRequest) (IO b) (IO b) ->
  Request IsoCodec meth path query hs LBS.ByteString ->
  Request IsoCodec meth path query hs b
body c r = r { body = IsoCodec c }

pathOf :: forall a meth q h b. (Generic a, GPath (Rep a))
    => Request IsoCodec meth [Text.Text] q h b -> Request IsoCodec meth a q h b
pathOf r = r { path = IsoCodec (Path.pathCodec @a) }

queryOf :: forall a meth p h b. (Generic a, GQuery (Rep a))
    => Request IsoCodec meth p HTTP.Query h b -> Request IsoCodec meth p a h b
queryOf r = r { query = IsoCodec (Query.queryCodec @a) }

headersOf :: forall a meth p q b. (Generic a, GHeaders ForRequest (Rep a))
    => Request IsoCodec meth p q [HTTP.Header] b -> Request IsoCodec meth p q a b
headersOf r = r { headers = IsoCodec (headersCodec @ForRequest @a) }

```



Okapi.Protocol.Request.Body

```
module Okapi.Protocol.Request.Body
  ( module Okapi.Protocol.Body
  , IsoJson
  , body
  ) where

import Okapi.Protocol.Body hiding (html)
import Okapi.Protocol.Request (body)
```

Okapi.Protocol.Request.Headers

```
module Okapi.Protocol.Request.Headers
  ( module Okapi.Protocol.Headers
  , headers
  ) where

import Okapi.Protocol.Headers hiding (setCookie)
import Okapi.Protocol.Request (headers)
```

Okapi.Protocol.Request.Method

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE StandaloneDeriving #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Request.Method (
  KnownMethod (..),
  Method,
  ParseError (..),
  parse,
  print,
  raw,
  method,
  knownMethodToStd,
  extractMethod,
  GET,
  POST,
  PUT,
  DELETE,
) where

import Control.Applicative ((<|>))
import GHC.TypeLits (Symbol)
import Network.HTTP.Types qualified as HTTP
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Prelude hiding (print)

data KnownMethod (m :: Symbol) where
  GET      :: KnownMethod "GET"
  POST     :: KnownMethod "POST"
  PUT      :: KnownMethod "PUT"
  DELETE   :: KnownMethod "DELETE"

deriving instance Eq   (KnownMethod m)
deriving instance Show (KnownMethod m)

type GET      = KnownMethod "GET"
type POST     = KnownMethod "POST"
type PUT      = KnownMethod "PUT"
type DELETE   = KnownMethod "DELETE"

data Method a where
  Raw      :: Method HTTP.Method
  Method   :: KnownMethod m -> Method (KnownMethod m)

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf Method = HTTP.Method
type instance ParseErrorOf Method = ParseError
```

```

parse :: Codec Method i o -> HTTP.Method -> (Either ParseError o, HTTP.Method)
parse = Codec.parser methodAlg
  where
    methodAlg :: forall a. Method a -> HTTP.Method -> (Either ParseError a, HTTP.Method)
    methodAlg Raw m = (Right m, m)
    methodAlg (Method km) m
      | m == HTTP.renderStdMethod (knownMethodToStd km) = (Right km, m)
      | otherwise = (Left ParseError, m)

print :: Codec Method i o -> i -> HTTP.Method
print = Codec.printer methodPrinter
  where
    methodPrinter :: forall a. Method a -> a -> HTTP.Method
    methodPrinter Raw m = m
    methodPrinter (Method km) _ = HTTP.renderStdMethod (knownMethodToStd km)

raw :: Codec Method HTTP.Method HTTP.Method
raw = Lift Raw

method :: KnownMethod m -> Codec Method (KnownMethod m) (KnownMethod m)
method km = Lift (Method km)

knownMethodToStd :: KnownMethod m -> HTTP.StdMethod
knownMethodToStd GET = HTTP.GET
knownMethodToStd POST = HTTP.POST
knownMethodToStd PUT = HTTP.PUT
knownMethodToStd DELETE = HTTP.DELETE

extractMethod :: Codec Method i o -> Maybe HTTP.StdMethod
extractMethod (Lift (Method km)) = Just (knownMethodToStd km)
extractMethod (Lift _) = Nothing
extractMethod (FMap _ c) = extractMethod c
extractMethod (LMap _ c) = extractMethod c
extractMethod (Apply cf cx) = extractMethod cf <|> extractMethod cx
extractMethod (Pure _) = Nothing

```

Okapi.Protocol.Request.Path

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Protocol.Request.Path (
    Path (..),
    ParseError (..),
    segment_,
    segment,
    segments,
    raw,
    parse,
    parseExact,
    print,
    LitF (..),
    GPath (..),
    pathCodec,
) where

import Data.Int (Int16, Int32, Int64)
import Data.Kind (Type)
import Data.List.NonEmpty (NonEmpty)
import Data.List.NonEmpty qualified as NEL
import Data.Proxy (Proxy (..))
import Data.Scientific (Scientific)
import Data.Text (Text)
import Data.Text qualified as Text
import Data.Time (Day, LocalTime, TimeOfDay, UTCTime)
import Data.UUID (UUID)
import GHC.Generics (C1, D1, Generic (..), K1 (..), M1 (..), Rec0, S1, Selector (..), (::*) (..))
import GHC.TypeLits (KnownSymbol, Symbol, symbolVal)
import Okapi.Codec (Codec (..), ParseErrorOf, PartOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data
    ( Data (..), Iso (..), Prim (..)
    , boolPrim, charPrim, dayPrim, doublePrim, floatPrim, int16Prim, int32Prim
    , int64Prim, intPrim, integerPrim, localTimePrim, scientificPrim, textPrim
    , timeOfDayPrim, utcTimePrim, uuidPrim
    )
import Prelude hiding (print)

data Path a where
    Seg_ :: Iso Path a -> a -> Path ()
    Seg :: Text -> Iso Path a -> Path a
    Segs :: Iso Path a -> Path (NonEmpty a)
```

```

Raw :: Path [Text]

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf Path = [Text]
type instance ParseErrorOf Path = ParseError
type instance PartOf Path = Text

parse :: Codec Path i o -> [Text] -> (Either ParseError o, [Text])
parse = Codec.parser pathAlg
  where
    pathAlg :: forall a. Path a -> [Text] -> (Either ParseError a, [Text])
    pathAlg (Seg_ vIso x) (t : ts)
      | t == vIso.encode x = (Right (), ts)
      | otherwise = (Left ParseError, t : ts)
    pathAlg (Seg_ _ _) [] = (Left ParseError, [])
    pathAlg (Seg_name vIso) (t : ts) = case vIso.decode t of
      Left _ -> (Left ParseError, t : ts)
      Right v -> (Right v, ts)
    pathAlg (Seg_ _ _) [] = (Left ParseError, [])
    pathAlg (Segs vIso) ts = case NEL.nonEmpty ts of
      Nothing -> (Left ParseError, [])
      Just nel -> case traverse vIso.decode (NEL.toList nel) of
        Left _ -> (Left ParseError, ts)
        Right xs -> case NEL.nonEmpty xs of
          Nothing -> (Left ParseError, [])
          Just nel' -> (Right nel', [])
    pathAlg Raw ts = (Right ts, [])

parseExact :: Codec Path i o -> [Text] -> Either (ParseError, [Text]) o
parseExact pathCodec' path = case parse pathCodec' path of
  (Left e, p) -> Left (e, p)
  (Right a, []) -> Right a
  (Right _, p) -> Left (ParseError, p)

print :: Codec Path i o -> i -> [Text]
print = Codec.printer pathPrinter
  where
    pathPrinter :: forall a. Path a -> a -> [Text]
    pathPrinter (Seg_ vIso x) () = [vIso.encode x]
    pathPrinter (Seg_name vIso) v = [vIso.encode v]
    pathPrinter (Segs vIso) nel = map vIso.encode (NEL.toList nel)
    pathPrinter Raw ts = ts

segment_ :: Iso Path a -> a -> Codec Path b ()
segment_ vIso x = Codec.LMap (const ()) (Lift (Seg_ vIso x))

segment :: Text -> Iso Path a -> Codec Path a a
segment name vIso = Lift (Seg name vIso)

segments :: Iso Path a -> Codec Path (NonEmpty a) (NonEmpty a)
segments vIso = Lift (Segs vIso)

```



```

raw :: Codec Path [Text] [Text]
raw = Lift Raw

pathIso :: Prim a -> Iso Path a
pathIso (Prim dec enc nfo) = Iso (either (const (Left ParseError)) Right . dec) enc nfo

instance Data Path Int      where iso = pathIso intPrim
instance Data Path Int16   where iso = pathIso int16Prim
instance Data Path Int32   where iso = pathIso int32Prim
instance Data Path Int64   where iso = pathIso int64Prim
instance Data Path Integer where iso = pathIso integerPrim
instance Data Path Float   where iso = pathIso floatPrim
instance Data Path Double  where iso = pathIso doublePrim
instance Data Path Scientific where iso = pathIso scientificPrim
instance Data Path Bool    where iso = pathIso boolPrim
instance Data Path Char    where iso = pathIso charPrim
instance Data Path Text    where iso = pathIso textPrim
instance Data Path Day     where iso = pathIso dayPrim
instance Data Path LocalTime where iso = pathIso localTimePrim
instance Data Path UTCTime where iso = pathIso utcTimePrim
instance Data Path TimeOfDay where iso = pathIso timeOfDayPrim
instance Data Path UUID    where iso = pathIso uuidPrim

data LitF (sym :: Symbol) = LitF deriving (Eq, Show)

class GPath (f :: Type -> Type) where
  gPathCodec :: Codec Path (f ()) (f ())

instance (GPath f) => GPath (D1 meta f) where
  gPathCodec = FMap M1 $ LMap unM1 gPathCodec

instance (GPath f) => GPath (C1 meta f) where
  gPathCodec = FMap M1 $ LMap unM1 gPathCodec

instance (GPath f, GPath g) => GPath (f :+: g) where
  gPathCodec =
    Apply
      (FMap (\l r -> l :+: r) (LMap (\(l :+: _) -> l) gPathCodec))
      (LMap (\(_ :+: r) -> r) gPathCodec)

instance {-# OVERLAPPING #-} (KnownSymbol sym) => GPath (S1 meta (Rec0 (LitF sym))) where
  gPathCodec =
    let txt = Text.pack (symbolVal (Proxy @sym))
    in FMap (\() -> M1 (K1 LitF)) $ LMap (const ()) $ Lift (Seg_ (iso @Path @Text) txt)

instance (Selector meta, Data Path a) => GPath (S1 meta (Rec0 a)) where
  gPathCodec =
    let name = Text.pack (selName (undefined :: S1 meta (Rec0 a) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Seg name (iso @Path @a))

pathCodec :: forall a. (Generic a, GPath (Rep a)) => Codec Path a a
pathCodec = FMap (to @a) $ LMap (from @a) gPathCodec

```

Okapi.Protocol.Request.Query

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Protocol.Request.Query (
    Query (..),
    ArrayStyle (..),
    ParseError (..),
    parse,
    print,
    raw,
    param,
    param',
    param_,
    flag,
    flag',
    list,
    list',
    GQuery (..),
    queryCodec,
) where

import Data.ByteString (ByteString)
import Data.ByteString qualified as BS
import Data.ByteString.Char8 qualified as BS8
import Data.Int (Int16, Int32, Int64)
import Data.Kind (Type)
import Data.List (partition)
import Data.List.NonEmpty (NonEmpty)
import Data.List.NonEmpty qualified as NE
import Data.Scientific (Scientific)
import Data.Text (Text)
import Data.Text qualified as Text
import Data.Text.Encoding (decodeUtf8Lenient, encodeUtf8)
import Data.Time (Day, LocalTime, TimeOfDay, UTCTime)
import Data.UUID (UUID)
import GHC.Generics (C1, D1, Generic (..), K1 (..), M1 (..), Rec0, S1, Selector (..), (:*:) (..))
import Network.HTTP.Types qualified as HTTP
import Okapi.Codec (Codec (..), ParseErrorOf, PartOf, StateOf)
import Okapi.Codec qualified as Codec
import Okapi.Data
    ( Data (..), Iso (..), Prim (..)
    , boolPrim, charPrim, dayPrim, doublePrim, floatPrim, int16Prim, int32Prim
    , int64Prim, intPrim, integerPrim, localTimePrim, scientificPrim, textPrim
```

```

    , timeOfDayPrim, utcTimePrim, uuidPrim
  )
import Prelude hiding (print)

type Query :: Type -> Type
data Query a where
  Raw :: Query HTTP.Query
  Param :: Text -> Iso Query a -> Query a
  Param' :: Text -> Iso Query a -> Query (Maybe a)
  Param_ :: Text -> Iso Query a -> a -> Query ()
  Flag :: Text -> Query ()
  Flag' :: Text -> Query Bool
  List :: ArrayStyle -> Text -> Iso Query a -> Query (NonEmpty a)
  List' :: ArrayStyle -> Text -> Iso Query a -> Query [a]

data ArrayStyle = Exploded | CommaDelimited | SpaceDelimited | PipeDelimited deriving (Eq, Show)

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf Query = HTTP.Query
type instance ParseErrorOf Query = ParseError
type instance PartOf Query = Text

parse :: Codec Query i o -> HTTP.Query -> (Either ParseError o, HTTP.Query)
parse = Codec.parser queryAlg
  where
    queryAlg :: forall a. Query a -> HTTP.Query -> (Either ParseError a, HTTP.Query)
    queryAlg Raw q = (Right q, [])
    queryAlg (Param key vIso) q =
      case partition \(k, _) -> k == encodeUtf8 key) q of
        ([], _) -> (Left ParseError, q)
        (_, Nothing) : _, _ -> (Left ParseError, q)
        (_, Just v) : _, rest -> case vIso.decode (decodeUtf8Lenient v) of
          Left e -> (Left e, q)
          Right x -> (Right x, rest)
    queryAlg (Param' key vIso) q =
      case partition \(k, _) -> k == encodeUtf8 key) q of
        ([], _) -> (Right Nothing, q)
        (_, Nothing) : _, rest -> (Right Nothing, rest)
        (_, Just v) : _, rest -> case vIso.decode (decodeUtf8Lenient v) of
          Left _ -> (Right Nothing, rest)
          Right x -> (Right (Just x), rest)
    queryAlg (Flag key) q =
      case partition \(k, _) -> k == encodeUtf8 key) q of
        ([], _) -> (Left ParseError, q)
        _ : _, rest -> (Right (), rest)
    queryAlg (Param_ key vIso x) q =
      case partition \(k, _) -> k == encodeUtf8 key) q of
        (_, Just v) : _, rest | decodeUtf8Lenient v == vIso.encode x -> (Right (), rest)
        _ -> (Left ParseError, q)
    queryAlg (Flag' key) q =
      case partition \(k, _) -> k == encodeUtf8 key) q of
        ([], _) -> (Right False, q)

```

```

    (_ : _, rest) -> (Right True, rest)
queryAlg (List style key vIso) q =
    let (vals, rest) = collectList style key q
    in case traverse (vIso.decode . decodeUtf8Lenient) vals of
        Left _ -> (Left ParseError, q)
        Right xs -> case NE.nonEmpty xs of
            Nothing -> (Left ParseError, q)
            Just nel -> (Right nel, rest)
queryAlg (List' style key vIso) q =
    let (vals, rest) = collectList style key q
    in case traverse (vIso.decode . decodeUtf8Lenient) vals of
        Left _ -> (Left ParseError, q)
        Right xs -> (Right xs, rest)

print :: Codec Query i o -> i -> HTTP.Query
print = Codec.printer queryPrinter
    where
        queryPrinter :: forall a. Query a -> a -> HTTP.Query
        queryPrinter Raw q = q
        queryPrinter (Param key vIso) x = [(encodeUtf8 key, Just (encodeUtf8 (vIso.encode x)))]
        queryPrinter (Param' _ _) Nothing = []
        queryPrinter (Param' key vIso) (Just x) = [(encodeUtf8 key, Just (encodeUtf8 (vIso.encode x)))]
        queryPrinter (Param_ key vIso x) () = [(encodeUtf8 key, Just (encodeUtf8 (vIso.encode x)))]
        queryPrinter (Flag key) () = [(encodeUtf8 key, Nothing)]
        queryPrinter (Flag' key) True = [(encodeUtf8 key, Nothing)]
        queryPrinter (Flag' _) False = []
        queryPrinter (List style key vIso) nel = renderList style key vIso (NE.toList nel)
        queryPrinter (List' style key vIso) xs = renderList style key vIso xs

collectList :: ArrayStyle -> Text -> HTTP.Query -> ([ByteString], HTTP.Query)
collectList Exploded key q =
    let k = encodeUtf8 key
        (matched, rest) = partition (\(ik, _) -> ik == k) q
    in ([v | (_, Just v) <- matched], rest)
collectList style key q =
    let k = encodeUtf8 key
    in case partition (\(ik, _) -> ik == k) q of
        ([], _) -> ([], q)
        (_, Nothing) : _, rest -> ([], rest)
        (_, Just v) : _, rest -> (filter (not . BS.null) (BS8.split (delim style) v), rest)

renderList :: ArrayStyle -> Text -> Iso Query a -> [a] -> HTTP.Query
renderList Exploded key vIso xs = [(encodeUtf8 key, Just (enc x)) | x <- xs]
    where
        enc = encodeUtf8 . vIso.encode
renderList style key vIso xs
    | null xs = []
    | otherwise = [(encodeUtf8 key, Just (BS.intercalate (BS8.singleton (delim style)) (map enc xs)))]
    where
        enc = encodeUtf8 . vIso.encode

delim :: ArrayStyle -> Char
delim CommaDelimited = ','

```

```

delim SpaceDelimited = ' '
delim PipeDelimited = '|'
delim Exploded = ','

raw :: Codec Query HTTP.Query HTTP.Query
raw = Lift Raw

param :: Text -> Iso Query a -> Codec Query a a
param key vIso = Lift (Param key vIso)

param' :: Text -> Iso Query a -> Codec Query (Maybe a) (Maybe a)
param' key vIso = Lift (Param' key vIso)

param_ :: Text -> Iso Query a -> a -> Codec Query () ()
param_ key vIso x = Lift (Param_ key vIso x)

flag :: Text -> Codec Query () ()
flag key = Lift (Flag key)

flag' :: Text -> Codec Query Bool Bool
flag' key = Lift (Flag' key)

list :: ArrayStyle -> Text -> Iso Query a -> Codec Query (NonEmpty a) (NonEmpty a)
list style key vIso = Lift (List style key vIso)

list' :: ArrayStyle -> Text -> Iso Query a -> Codec Query [a] [a]
list' style key vIso = Lift (List' style key vIso)

queryIso :: Prim a -> Iso Query a
queryIso (Prim dec enc nfo) = Iso (either (const (Left ParseError)) Right . dec) enc nfo

instance Data Query Int      where iso = queryIso intPrim
instance Data Query Int16   where iso = queryIso int16Prim
instance Data Query Int32   where iso = queryIso int32Prim
instance Data Query Int64   where iso = queryIso int64Prim
instance Data Query Integer where iso = queryIso integerPrim
instance Data Query Float   where iso = queryIso floatPrim
instance Data Query Double  where iso = queryIso doublePrim
instance Data Query Scientific where iso = queryIso scientificPrim
instance Data Query Bool    where iso = queryIso boolPrim
instance Data Query Char    where iso = queryIso charPrim
instance Data Query Text    where iso = queryIso textPrim
instance Data Query Day     where iso = queryIso dayPrim
instance Data Query LocalTime where iso = queryIso localTimePrim
instance Data Query UTCTime where iso = queryIso utcTimePrim
instance Data Query TimeOfDay where iso = queryIso timeOfDayPrim
instance Data Query UUID    where iso = queryIso uuidPrim

class GQuery (f :: Type -> Type) where
  gQueryCodec :: Codec Query (f ()) (f ())

instance (GQuery f) => GQuery (D1 meta f) where
  gQueryCodec = FMap M1 $ LMap unM1 gQueryCodec

```

```

instance (GQuery f) => GQuery (C1 meta f) where
  gQueryCodec = FMap M1 $ LMap unM1 gQueryCodec

instance (GQuery f, GQuery g) => GQuery (f :*: g) where
  gQueryCodec =
    Apply
      (FMap (\l r -> l :*: r) (LMap (\(l :*: _) -> l) gQueryCodec))
      (LMap (\(_ :*: r) -> r) gQueryCodec)

instance (Selector s, Data Query a) => GQuery (S1 s (Rec0 a)) where
  gQueryCodec =
    let key = Text.pack (selName (undefined :: S1 s (Rec0 a) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Param key (iso @Query @a))

instance {-# OVERLAPPING #-} (Selector s, Data Query a) => GQuery (S1 s (Rec0 (Maybe a))) where
  gQueryCodec =
    let key = Text.pack (selName (undefined :: S1 s (Rec0 (Maybe a) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Param' key (iso @Query @a))

instance {-# OVERLAPPING #-} (Selector s) => GQuery (S1 s (Rec0 ())) where
  gQueryCodec =
    let key = Text.pack (selName (undefined :: S1 s (Rec0 ())) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Flag key)

instance {-# OVERLAPPING #-} (Selector s) => GQuery (S1 s (Rec0 Bool)) where
  gQueryCodec =
    let key = Text.pack (selName (undefined :: S1 s (Rec0 Bool) ()))
    in FMap (M1 . K1) $ LMap (unK1 . unM1) $ Lift (Flag' key)

queryCodec :: forall a. (Generic a, GQuery (Rep a)) => Codec Query a a
queryCodec = FMap (to @a) $ LMap (from @a) gQueryCodec

```

Okapi.Protocol.Response

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoFieldSelectors #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE ScopedTypeVariables #-}
{-# LANGUAGE TypeApplications #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

module Okapi.Protocol.Response where

import Data.ByteString.Builder qualified as Builder
import Data.ByteString.Lazy qualified as LBS
import Data.Kind (Type)
import Data.List.NonEmpty (NonEmpty (..))
import GHC.Generics (C1, D1, Generic (..), K1 (..), M1 (..), Rec0, Rep, S1, (:+:) (..))
import Network.HTTP.Types qualified as HTTP
import Network.Wai qualified as Wai
import Network.Wai.Internal qualified as WaiI
import Okapi.Codec (Codec, ForResponse, IsoCodec (..), ParseError (..), Result (..), Value (..))
import Okapi.Protocol.Body (Body)
import Okapi.Protocol.Body qualified as Body
import Okapi.Protocol.Headers (Headers)
import Okapi.Protocol.Headers qualified as Headers
import Okapi.Protocol.Response.Status (Status, S200, S201, S204, S404, S500)
import Okapi.Protocol.Response.Status qualified as Status

data Response (mode :: (Type -> Type) -> Type -> Type) status headers body = Response
  { status  :: mode Status status
  , headers :: mode (Headers ForResponse) headers
  , body    :: mode (Body ForResponse) (IO body)
  }

response :: Response IsoCodec HTTP.Status [HTTP.Header] LBS.ByteString
response = Response
  { status  = IsoCodec Status.raw
  , headers = IsoCodec Headers.raw
  , body    = IsoCodec Body.raw
  }

responseValue :: status -> headers -> IO body -> Response Value status headers body
responseValue s h b = Response
  { status  = Value s
  , headers = Value h
  , body    = Value b
  }
```

```

status200 :: Response IsoCodec S200 [HTTP.Header] LBS.ByteString
status200 = Response
  { status = IsoCodec (Status.status Status.S200)
  , headers = IsoCodec Headers.raw
  , body = IsoCodec Body.raw
  }

status201 :: Response IsoCodec S201 [HTTP.Header] LBS.ByteString
status201 = Response
  { status = IsoCodec (Status.status Status.S201)
  , headers = IsoCodec Headers.raw
  , body = IsoCodec Body.raw
  }

status204 :: Response IsoCodec S204 [HTTP.Header] LBS.ByteString
status204 = Response
  { status = IsoCodec (Status.status Status.S204)
  , headers = IsoCodec Headers.raw
  , body = IsoCodec Body.raw
  }

status404 :: Response IsoCodec S404 [HTTP.Header] LBS.ByteString
status404 = Response
  { status = IsoCodec (Status.status Status.S404)
  , headers = IsoCodec Headers.raw
  , body = IsoCodec Body.raw
  }

status500 :: Response IsoCodec S500 [HTTP.Header] LBS.ByteString
status500 = Response
  { status = IsoCodec (Status.status Status.S500)
  , headers = IsoCodec Headers.raw
  , body = IsoCodec Body.raw
  }

headers ::
  Codec (Headers ForResponse) h h ->
  Response IsoCodec status [HTTP.Header] bdy ->
  Response IsoCodec status h bdy
headers c r = r { headers = IsoCodec c }

body ::
  Codec (Body ForResponse) (IO b) (IO b) ->
  Response IsoCodec status hs LBS.ByteString ->
  Response IsoCodec status hs b
body c r = r { body = IsoCodec c }

extractWaiResBody :: Wai.Response -> LBS.ByteString
extractWaiResBody (WaiI.ResponseBuilder _ _ b) = Builder.toLazyByteString b
extractWaiResBody _ = LBS.empty

parseResponseResult :: Response IsoCodec status headers body -> Wai.Response -> IO (Response Result status headers
parseResponseResult codec waiRes = do

```



```

let status = Wai.responseStatus waiRes
    hdrs    = Wai.responseHeaders waiRes
    bodyLbs = extractWaiResBody waiRes
    (sr, _) = Status.parse codec.status.isoCodec status
    (hr, _) = Headers.parse codec.headers.isoCodec hdrs
    (br, _) = Body.parse codec.body.isoCodec bodyLbs
pure Response
    { status = Result sr
    , headers = Result hr
    , body = Result br
    }

resultToValue :: Response Result status headers body -> Maybe (Response Value status headers body)
resultToValue r = case (r.status.result, r.headers.result, r.body.result) of
    (Right s, Right h, Right b) -> Just (responseValue s h b)
    _                            -> Nothing

resultToParseError :: Response Result status headers body -> Response ParseError status headers body
resultToParseError r = Response
    { status = ParseError (either Just (const Nothing) r.status.result)
    , headers = ParseError (either Just (const Nothing) r.headers.result)
    , body = ParseError (either Just (const Nothing) r.body.result)
    }

printOne :: Response IsoCodec status headers body -> Response Value status headers body -> IO Wai.Response
printOne codec rv = do
    bodyBytes <- Body.printM codec.body.isoCodec rv.body.value
    let hdrs = Headers.print codec.headers.isoCodec rv.headers.value
    pure (Wai.responseLBS
        (Status.print codec.status.isoCodec rv.status.value)
        hdrs
        bodyBytes)

newtype Responses
    (mode :: (Type -> Type) -> Type -> Type)
    (responses :: ((Type -> Type) -> Type -> Type) -> Type)
    = Responses (NonEmpty (responses mode))

getResponses :: Responses mode responses -> NonEmpty (responses mode)
getResponses (Responses xs) = xs

type family GResponseFunc (f :: Type -> Type) (res :: Type) :: Type where
    GResponseFunc (D1 meta f) res = GResponseFunc f res
    GResponseFunc (C1 meta f) res = GResponseFunc f res
    GResponseFunc (S1 meta (Rec0 (Response IsoCodec status headers body))) res = Response IsoCodec status headers body
    GResponseFunc (f :+: g) res = GResponseFunc f (GResponseFunc g res)

class GTraverse
    (f :: (Type -> Type) -> Type -> Type)
    (g :: (Type -> Type) -> Type -> Type)
    (fi :: Type -> Type)
    (fg :: Type -> Type) where
    gtraverse :: Functor t => (forall status headers body. Response f status headers body -> t (Response g status headers body))

```

```

instance GTraverse f g fi fg => GTraverse f g (D1 meta fi) (D1 meta fg) where
  gtraverse k (M1 x) = M1 <$> gtraverse k x

instance GTraverse f g fi fg => GTraverse f g (C1 meta fi) (C1 meta fg) where
  gtraverse k (M1 x) = M1 <$> gtraverse k x

instance GTraverse f g (S1 meta (Rec0 (Response f status headers body))) (S1 meta (Rec0 (Response g status headers body))) where
  gtraverse k (M1 (K1 resp)) = M1 . K1 <$> k resp

instance (GTraverse f g fil fgl, GTraverse f g fir fgr)
=> GTraverse f g (fil :+: fir) (fgl :+: fgr) where
  gtraverse k (L1 x) = L1 <$> gtraverse k x
  gtraverse k (R1 x) = R1 <$> gtraverse k x

traverseResponses ::
  forall f g responses t.
  (Generic (responses f), Generic (responses g), GTraverse f g (Rep (responses f)) (Rep (responses g)), Functor (Response f status headers body) => Response f status headers body -> t (Response g status headers body)) -> responses t
traverseResponses k = fmap to . gtraverse k . from

class GZip
  (f :: (Type -> Type) -> Type -> Type)
  (g :: (Type -> Type) -> Type -> Type)
  (fi :: Type -> Type)
  (fg :: Type -> Type) where
  gzip :: (forall status headers body. Response f status headers body -> Response g status headers body -> c) -> c

instance GZip f g fi fg => GZip f g (D1 meta fi) (D1 meta fg) where
  gzip k (M1 a) (M1 b) = gzip k a b

instance GZip f g fi fg => GZip f g (C1 meta fi) (C1 meta fg) where
  gzip k (M1 a) (M1 b) = gzip k a b

instance GZip f g (S1 meta (Rec0 (Response f status headers body))) (S1 meta (Rec0 (Response g status headers body))) where
  gzip k (M1 (K1 a)) (M1 (K1 b)) = Just (k a b)

instance (GZip f g fil fgl, GZip f g fir fgr)
=> GZip f g (fil :+: fir) (fgl :+: fgr) where
  gzip k (L1 a) (L1 b) = gzip k a b
  gzip k (R1 a) (R1 b) = gzip k a b
  gzip _ _ _ = Nothing

zipResponses ::
  forall f g responses c.
  (Generic (responses f), Generic (responses g), GZip f g (Rep (responses f)) (Rep (responses g))) =>
  (forall status headers body. Response f status headers body -> Response g status headers body -> c) -> responses c
zipResponses k a b = gzip k (from a) (from b)

class GConstruct (f :: Type -> Type) where
  gConstruct ::
    forall responses res. Generic (responses IsoCodec) =>
    (f () -> Rep (responses IsoCodec) ()) -> (NonEmpty (responses IsoCodec) -> res) -> GResponseFunc f res

```

```

instance GConstruct fi => GConstruct (D1 meta fi) where
  gConstruct inject cont = gConstruct (inject . M1) cont

instance GConstruct fi => GConstruct (C1 meta fi) where
  gConstruct inject cont = gConstruct (inject . M1) cont

instance GConstruct (S1 meta (Rec0 (Response IsoCodec status headers body))) where
  gConstruct inject cont codec = cont (to (inject (M1 (K1 codec))) :| [])

instance (GConstruct fil, GConstruct fir) => GConstruct (fil :+: fir) where
  gConstruct inject cont =
    gConstruct (inject . L1) $ \ls ->
    gConstruct (inject . R1) $ \rs ->
    cont (ls <> rs)

class
  ( Generic (responses IsoCodec), Generic (responses Result), Generic (responses Value), Generic (responses ParseError)
  , GConstruct (Rep (responses IsoCodec))
  , GTraverse IsoCodec Result      (Rep (responses IsoCodec)) (Rep (responses Result))
  , GTraverse Result Value        (Rep (responses Result))   (Rep (responses Value))
  , GTraverse Result ParseError   (Rep (responses Result))   (Rep (responses ParseError))
  , GTraverse IsoCodec IsoCodec   (Rep (responses IsoCodec)) (Rep (responses IsoCodec))
  , GZip      IsoCodec Value      (Rep (responses IsoCodec)) (Rep (responses Value))
  ) =>
  Cases (responses :: ((Type -> Type) -> Type -> Type) -> Type)
  where

  cases :: GResponseFunc (Rep (responses IsoCodec)) (Responses IsoCodec responses)
  cases = gConstruct id (Responses @IsoCodec @responses)

```

Okapi.Protocol.Response.Body

```
module Okapi.Protocol.Response.Body
  ( module Okapi.Protocol.Body
  , IsoJson
  , body
  ) where

import Okapi.Protocol.Body hiding (form)
import Okapi.Protocol.Response (body)
```

Okapi.Protocol.Response.Headers

```
module Okapi.Protocol.Response.Headers
  ( module Okapi.Protocol.Headers
  , module Okapi.Protocol.Headers.Attributes
  , headers
  ) where

import Okapi.Protocol.Headers hiding (cookie, cookie')
import Okapi.Protocol.Headers.Attributes hiding (raw, parse, print)
import Okapi.Protocol.Response (headers)
```

Okapi.Protocol.Response.Status

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE StandaloneDeriving #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}

module Okapi.Protocol.Response.Status (
    KnownStatus (..),
    Status,
    ParseError (..),
    parse,
    print,
    raw,
    status,
    knownStatusToHTTP,
    extractStatus,
    S200,
    S201,
    S204,
    S404,
    S500,
) where

import Control.Applicative ((<|>))
import Data.Kind (Type)
import GHC.TypeLits (Nat)
import Network.HTTP.Types qualified as HTTP
import Okapi.Codec (Codec (..), ParseErrorOf, StateOf)
import Okapi.Codec qualified as Codec
import Prelude hiding (print)

data KnownStatus (s :: Nat) where
    S200 :: KnownStatus 200
    S201 :: KnownStatus 201
    S204 :: KnownStatus 204
    S404 :: KnownStatus 404
    S500 :: KnownStatus 500

deriving instance Eq (KnownStatus s)
deriving instance Show (KnownStatus s)

type S200 = KnownStatus 200
type S201 = KnownStatus 201
type S204 = KnownStatus 204
type S404 = KnownStatus 404
type S500 = KnownStatus 500

instance Num (KnownStatus 200) where
    fromInteger 200 = S200
    fromInteger _ = error "KnownStatus 200: expected 200"
    S200 + S200 = S200
```

```

S200 * S200 = S200
abs = id; signum _ = S200; negate = id

instance Num (KnownStatus 201) where
  fromInteger 201 = S201
  fromInteger _ = error "KnownStatus 201: expected 201"
  S201 + S201 = S201
  S201 * S201 = S201
  abs = id; signum _ = S201; negate = id

instance Num (KnownStatus 204) where
  fromInteger 204 = S204
  fromInteger _ = error "KnownStatus 204: expected 204"
  S204 + S204 = S204
  S204 * S204 = S204
  abs = id; signum _ = S204; negate = id

instance Num (KnownStatus 404) where
  fromInteger 404 = S404
  fromInteger _ = error "KnownStatus 404: expected 404"
  S404 + S404 = S404
  S404 * S404 = S404
  abs = id; signum _ = S404; negate = id

instance Num (KnownStatus 500) where
  fromInteger 500 = S500
  fromInteger _ = error "KnownStatus 500: expected 500"
  S500 + S500 = S500
  S500 * S500 = S500
  abs = id; signum _ = S500; negate = id

type Status :: Type -> Type
data Status a where
  Raw    :: Status HTTP.Status
  Status :: KnownStatus s -> Status (KnownStatus s)

data ParseError = ParseError deriving (Eq, Show)

type instance StateOf Status = HTTP.Status
type instance ParseErrorOf Status = ParseError

parse :: Codec Status i o -> HTTP.Status -> (Either ParseError o, HTTP.Status)
parse = Codec.parser statusAlg
  where
    statusAlg :: forall a. Status a -> HTTP.Status -> (Either ParseError a, HTTP.Status)
    statusAlg Raw      s = (Right s, s)
    statusAlg (Status ks) s
      | s == knownStatusToHTTP ks = (Right ks, s)
      | otherwise                  = (Left ParseError, s)

print :: Codec Status i o -> i -> HTTP.Status
print = go
  where

```

```

go :: forall i' o'. Codec Status i' o' -> i' -> HTTP.Status
go (Lift (Status ks)) _ = knownStatusToHTTP ks
go (Lift Raw)          s = s
go (FMap _ c)          i = go c i
go (LMap f c)          i = go c (f i)
go (Apply cf _)        i = go cf i
go (Pure _)            _ = HTTP.status200

raw :: Codec Status HTTP.Status HTTP.Status
raw = Lift Raw

status :: KnownStatus s -> Codec Status (KnownStatus s) (KnownStatus s)
status ks = Lift (Status ks)

knownStatusToHTTP :: KnownStatus s -> HTTP.Status
knownStatusToHTTP S200 = HTTP.status200
knownStatusToHTTP S201 = HTTP.status201
knownStatusToHTTP S204 = HTTP.status204
knownStatusToHTTP S404 = HTTP.status404
knownStatusToHTTP S500 = HTTP.status500

extractStatus :: Codec Status i o -> Maybe HTTP.Status
extractStatus (Lift (Status ks)) = Just (knownStatusToHTTP ks)
extractStatus (Lift Raw)         = Nothing
extractStatus (FMap _ c)         = extractStatus c
extractStatus (LMap _ c)         = extractStatus c
extractStatus (Apply cf cx)      = extractStatus cf <|> extractStatus cx
extractStatus (Pure _)           = Nothing

```


Plan: Remove Prim

Context

Prim is a redundant intermediate type in `Okapi.Data` — a text-centric codec that all DSLs bridge into `Iso t a` via helpers like `headerIso`, `cookieIso`, `pathIso`. The whole point of `Iso t a` is that `decode :: PartOf t -> Either (ParseErrorOf t) a` is native to each DSL. Prim ignores this by routing everything through `Text` regardless of `PartOf t`, then converting back. Additionally, `numPrim` uses `readMaybe` (Read instances) for parsing — fragile and imprecise for floats.

The fix: delete `Prim` and all `*Prim` helpers, implement `Data t a` instances directly against each DSL's native `PartOf t`, using `http-api-data` (already a dependency) for proper parsing and encoding.

PartOf map

DSL	PartOf	ParseErrorOf
Path	Text	ParseError
Query	Text	ParseError
Headers ctx	ByteString	ParseError
Cookie	ByteString	ParseError

http-api-data API to use — per DSL

DSL	decode	encode
Path	<code>parseUrlPiece</code>	<code>toUrlPiece</code>
Query	<code>parseQueryParam</code>	<code>toQueryParam</code>
Headers ctx	<code>parseHeader</code>	<code>toHeader</code>
Cookie	<code>parseHeader</code>	<code>toHeader</code>

Error mapping: first `(const ParseError)` on the decode side.

Instance pattern

Path:

```
instance Data Path Int where
  iso = Iso (first (const ParseError) . parseUrlPiece) toUrlPiece (Info "integer" Nothing)
```

Query:

```
instance Data Query Int where
  iso = Iso (first (const ParseError) . parseQueryParam) toQueryParam (Info "integer" Nothing)
```

Headers / Cookie:

```
instance Data (Headers ctx) Int where
  iso = Iso (first (const ParseError) . parseHeader) toHeader (Info "integer" Nothing)
```

Exceptions not covered by `http-api-data` (keep custom implementations): - `Data (Headers ctx) ByteString` — identity - `Data (Headers ctx) DiffTime` — custom numeric format - `Data (Headers ctx) (TimeOfDay, TimeZone)` — custom format - `Data Cookie ByteString` — identity

Files to change

Okapi/Data.hs: Delete `Prim (..)`, `numPrim`, `timePrim`, and all `*Prim` values and their exports. Keep `Iso`, `Info`, `Data`, and the convenience `int/bool/etc.` helpers (those are still useful).

Okapi/Protocol/Request/Path.hs: Delete `pathIso`. Rewrite all `Data Path a` instances using `parseUrlPiece/toUrlPiece`. Add import `Web.HttpApiData`.

Okapi/Protocol/Request/Query.hs: Rewrite all `Data Query a` instances using `parseQueryParam/toQueryParam`. Add import `Web.HttpApiData`. Remove `Prim` imports from `Okapi.Data`.

Okapi/Protocol/Headers.hs: Delete `headerIso`. Rewrite `Data (Headers ctx) a` instances using `parseHeader/toHeader`. Keep custom instances for `ByteString`, `DiffTime`, `(TimeOfDay, TimeZone)`. Remove `Prim` imports from `Okapi.Data`.

Okapi/Protocol/Headers/Cookies.hs: Delete `cookieIso`. Rewrite `Data Cookie a` instances using `parseHeader/toHeader`. Keep custom `ByteString` instance. Remove all `Prim` imports.

Future work

`http-api-data` is a pragmatic stepping stone — its single `FromHttpApiData/ToHttpApiData` typeclass conflates path, query, header, and cookie wire formats which have genuinely different semantics (percent-encoding vs form-encoding vs RFC 7231 header format). Each DSL should eventually have its own hand-written parse/print logic. This plan just gets `Prim` deleted and `readMaybe` out of the picture.

Verification

```
cd lib && cabal build
```